

Slot-Chain: A Preconfirmation Protocol

Built on Per-Slot Signature Chains

v2 Design Specification

August 2026

Abstract

This document specifies the successor design (v2) of the Taiko preconfirmation protocol. L2 time is divided into one-second slots; a lookahead schedule, published an epoch in advance, assigns each slot to a unique builder. The builder produces a block and signs it, and because each signature covers the parent hash, the not-yet-landed blocks form a *signature chain*. Submitting such a chain to L1 together with a validity proof is called *landing*: an aggregator elected by a perpetual auction performs it in the normal case, but anyone may perform it when the aggregator is absent, because a block's authority comes from its builder's signature rather than from whoever lands it. Finality is given by the proof system together with a deterministic *settlement window*: among the candidates that have landed and been proven, the heaviest one is finalized when the window closes at a predetermined L1 height. We give the trust model, the liveness accounting, the slashing rules and the parameter geometry, and we state the accepted residual risks and the remaining open items explicitly rather than eliding them — including the deliberate absence of a censorship-resistance floor, which this design does not provide.

Contents

0	One-page summary	3
1	Design goals and non-goals	4
2	Roles	7
3	Time and scheduling	8
3.1	Slots and epochs	8
3.2	The lookahead	8
4	Builders	10
4.1	Registry	10
4.2	Block production and signing	11
4.3	Equivocation slashing (verified directly on L1)	14
5	The signature chain: validity, canonicity, preconfirmation semantics	14
5.1	Single-block validity (circuit rules)	14

5.2	Canonicity and fork choice	15
5.3	Gaps	17
5.4	Deletion vs. gaps: what the signature chain fixes and what remains (stated explicitly)	18
5.5	Signing the header without publishing the body (withhold-body) — a known non-slashable misbehavior	20
5.6	Settlement windows and heaviest-proven-batch finality (settlement-window finality) — mechanical, DA-free tail protection	21
6	Landing	23
6.1	Batches	23
6.2	Cadence	25
6.3	Landing SLA and Versioned Permissionless Recovery	26
6.4	Stall and Gap Recovery	28
6.5	Aggregator Seat and Economics	28
7	Anchoring and the bridge ($L1 \rightarrow L2$)	29
8	What happens when each role goes offline (liveness accounting, in a single table)	32
9	Master table of slashing and bonds	33
10	Detailed comparison with v15 (for readers who have read v15)	34
11	Implementation gates and fixed decisions	35
	Master parameter table	36
	Appendix A: Design alternatives considered and rejected	38
	Appendix B: Glossary	39
	Appendix C: Executable consensus models	40

Scope. This document specifies the Taiko preconfirmation protocol: L2 time is divided into one-second slots, a lookahead schedule assigns each slot to a single builder, each builder signs the block it produces over the parent hash so that unlanded blocks form a signature chain, and that chain is carried to L1 with a validity proof. It supersedes an earlier design line built on a perpetual auction with epoch-level determination, which §10 compares against.

How to read it. §0–§11 with Appendices A–C are the specification proper and can be read in order. A reader after the trunk alone can get a full picture from the §0 one-page summary together with the §8 liveness accounting table. Residual risks and accepted costs are stated where they arise rather than collected at the end; open items are listed in §11.

0 One-page summary

In one sentence: L2 has one slot per second, and the lookahead — which states "who holds the right to produce a block in this second" — is published two epochs in advance. The builder whose turn it is produces a block and signs it, and the signature covers the parent block hash; the blocks, which are not yet on chain, are thereby strung into a signature chain. The act of packing this chain onto L1 together with a validity proof is called landing; in normal operation the aggregator produced by the auction is responsible for it, but when the aggregator goes offline anyone may do it — because a block's authority comes from the builder's signature, not from the lander.

The complete path from block production to finality:

This structure solves three things at once:

1. **Second-scale liveness.** A builder going offline costs only its own slot (about 1 second), and the next builder simply continues producing blocks on top of the last visible chain head. There is no window in which "one participant goes offline and the chain stops for tens of minutes".
2. **No L2 consensus is required.** "Who should produce the block in this slot" is decided by the lookahead, and "whether this block is legal" is decided by pure-function rules inside the proving circuit (verify the signature, verify the lookahead assignment, verify the link) — anyone reading L1 computes the same answer, with no voting and no arbiter.
3. **Landing carries no privilege, and the aggregator is not a single point of failure.** The signature chain carries its own authority; landing is merely transport. If the designated aggregator times out without landing, anyone may pick the signature chain up, land it, and collect a reward from the aggregator-funded reward escrow (§6.3).

What is kept and what is removed (relative to v15):

Kept	Removed
The perpetual auction skeleton for the aggregator seat	Epoch boundary commitments (EBC) and the once-per-epoch irreversible determination
The per-block signature chain and the two-tier parent rule (new here, not inherited from v15)	The default derivation rule (§6.8 default derivation)

Kept	Removed
—	The forced-inclusion queue — removed wholesale by the design decision (simplification), together with forced-only blocks, <code>P_forced</code> , the dual queues and <code>C_force/C_bridge/F_delay/H_force</code>
The "party at fault pays, permissionless substitution" model (fault-paid permissionless fallback)	Total Anarchy mode (§8) — landing itself can already be backstopped permissionlessly, so it is no longer needed
The tiered bond / slashing concept	<code>K_empty</code> , availability certificates (AC), the <code>H_cancel</code> cancellation cascade, and the rest of the machinery supporting the epoch determination
The per-block anchor approach to L1→L2 bridging	Handover and seat-switching arbitration mechanisms (replaced by circuit rules)

Timeline illustration (slot = 1 second, epoch = 384 slots) — going offline leaves only a gap, and the chain does not stop:

The relationship between landing and fallback (details in §6): normal candidates compete by the heaviest-proven order until a deterministic close. If final lag breaches the strict SLA, the incumbent is terminated, the normal window is either committed if already qualifying or canceled, and one persistent fallback episode accepts the first episode-bound proof meeting the acceptance-time progress target. Proof and data costs are paid from the outgoing seat's separate pre-funded escrow; the penalty bond is burned (§6.3).

1 Design goals and non-goals

Goals:

- **[G1] Second-scale continuity of ordering.** If any single participant (a builder or the aggregator) goes offline, the interruption of ordering service perceptible to users lasts at most a few slots (seconds), not a few epochs (minutes to hours).
- **[G2] No L2 consensus.** The lookahead, legality and canonicity are all pure functions of L1 data and circuit rules; L2 nodes neither vote nor arbitrate among themselves. This inherits the spirit of v15's I1: derivation is a deterministic function of L1 data.
- **[G3] Landing carries no privilege.** Moving an already-signed chain onto L1 requires no identity whatsoever; the designated aggregator is merely an efficiency arrangement (avoiding gas competition and duplicated proving), not an arrangement of power.
- **[G4] Proof plus a deterministic window is finality.** A landed batch carries a validity proof, which L1 verifies on the spot, and the batch becomes a candidate; at the close of the settlement window the heaviest proven candidate is finalized — there is still no optimistic period, no fraud proof and no arbitration, and there is no intermediate "proposed but unproven" state, so the entire body of machinery built around such an intermediate state (sealing deadlines, cancellation cascades, and most of the timing exemptions) is unnecessary.
- **[G5] (withdrawn) An inclusion floor is no longer a goal of this design.** There is no builder-independent entry path. If every scheduled builder censors an address, the protocol offers no remedy and no independent exit. The all-cartel row of §8 records the resulting unbounded failure.
- **[G6] A low barrier to entry — only in a weakened form.** A builder funds one independent

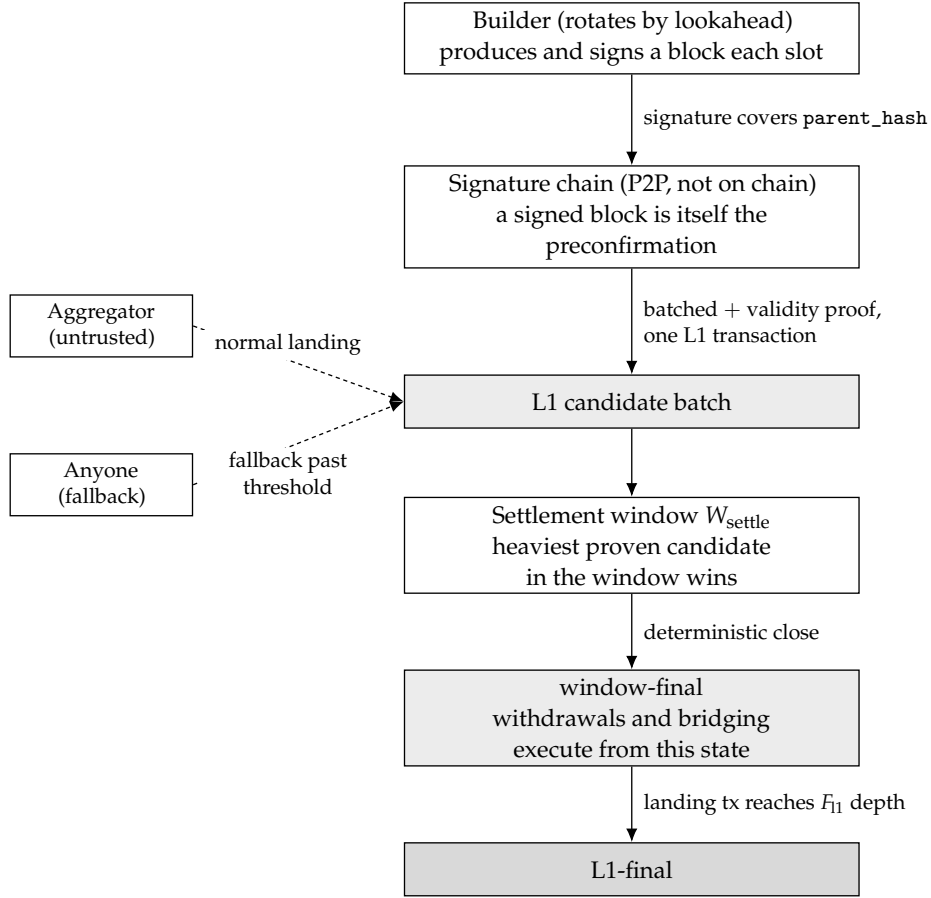


Figure 1: The complete path from block production to finality: builder signatures produce a signature chain that never touches the chain, anyone may land it on L1 together with a validity proof to become a candidate, and once the settlement window closes it becomes window-final and then L1-final. Landing carries no privilege, so the aggregator and the fallback lander are interchangeable.

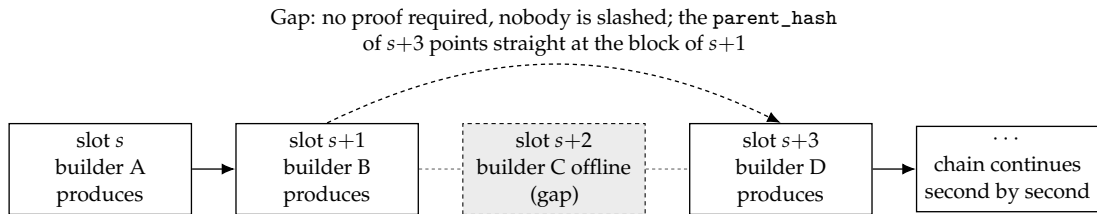


Figure 2: A single builder going offline leaves nothing but a one-slot gap in the timeline: the next builder takes the last visible block as its parent and carries on, so the chain never stalls.

L_{lease} tranche for every schedule window in which it accepts tickets. Each tranche prices at most Q_{max}=76 slots; the total locked amount is the number of simultaneously reserved future and still-liable historical windows times that tranche. This is materially more than one slot's value and must be published as an explicit capital requirement, not described as permissionless by assertion.

Trust model (normative):

- **Builders: honest majority.** We assume that within each lookahead window (on the order of an epoch), most of the scheduled builders honestly follow the protocol (producing blocks on time, broadcasting block headers and block bodies, and extending the highest available chain head).
- **Aggregators: untrusted.** An aggregator may misbehave arbitrarily — delaying, truncating, choosing among forks, colluding with a minority of builders — and the protocol must, under such conditions, still hold the bounds given in the §8 table. The scope of those bounds: the bounds in §8 are liveness bounds (advancing finality, seat turnover); the exposure of preconfirmation safety to collusion between the aggregator and malicious builders — isolation without slashing, a single-block deep skip of depth $\leq G_{\max} - 1$, and a minority coalition relaying a sparse branch to a depth that can reach the length of the unlanded tail ($\approx \Delta_{\text{lag}} + \text{fallback response}$) — is the safety residual risk explicitly priced in §5.4, and it falls outside the "bounded" promise.
- **L1: safe and live.** A sufficiently funded transaction is assumed included within T_{include,max} L1 slots. Setter relations require enough time both before a normal close and inside D_{anchor,max}; fallback uses one persistent first-valid episode rather than timed rounds. Targeted censorship longer than this bound is out-of-model and may delay recovery or force a recovery block to be re-signed.

The honest majority is an economic and social assumption, not one enforced by the protocol : the draw is weighted by bond and w_{max} is no defense against Sybils (§3.2), so a single capital holder could in principle buy up the majority of the weight and push the system out of the model — of the same kind as the honest-majority assumption of any PoS chain. This design does not claim that the builder role is entirely trustless or entirely permissionless; this is the trust model explicitly accepted (2026-08-25), and what is bought in exchange is not paying the cost of protocol-level DA to cover collusion by all participants (the cost identity is given in Appendix A-3). Each promise is calibrated accordingly: the in-model bounds are given in the §8 table; out-of-model, With no forced inclusion, nothing is guaranteed. A note on the acceptance criterion: if the acceptance criterion is "no role may be trusted, everything fully permissionless", this design neither satisfies it nor attempts to — the acceptance criterion is the trust model accepted in this section; the user-facing promises, graded into three tiers, are stated in §5.4.

How much of the assumption each mechanism actually consumes (written out, to keep the assumption from being quietly strengthened):

- Fallback assumes at least one honest scheduled ticket in every fixed window. Activation just after that window's last honest ticket can wait through the rest of this window and almost all of the next, so H_{recovery,max}=2×W_{size}; this wait is included in Δ_{close} rather than silently assigned to the prover;

- The frequency with which preconfirmations are isolated (skips, body withholding) is bounded by the share of dishonest builders — this is where "majority" is used, and it is load-bearing there: skipping is profitable for the skipper, not a behavior that extinguishes itself for being unprofitable;
- Collusion of the entire builder set together with the aggregator (block bodies kept private, the chain landable by no one) lies outside the model: the protocol provides no finality guarantee in that case. After This design does not include forced inclusion, this case has **no defense in depth left**: the former guarantee that "even if the assumption is broken, forced transactions and bridge messages still operate independently on L1 data alone" no longer holds, and a user's inclusion and exit depend entirely on the honesty of the scheduled builders;
- **Preconfirmation safety for the tail additionally depends on the completeness of the lander's view** : a lander that is honest but eclipsed or blinded will land the wrong tail and revoke the preconfirmations of a heavier tail that it never saw. The mitigation is "landing depth $\geq$ propagation settling" (§5.2), which compresses this risk down to "equivocation (slashable) or eclipse (the P2P layer)"; together with "the honest majority is an economic assumption" and "liveness under a Byzantine aggregator is conditional on fallback being economically viable", it belongs to the explicitly stated non-pure-protocol dependencies.

Non-goals (not addressed in this version):

- Per-transaction fair exchange: the preconfirmation receipt is block-level (§5).
- User restitution: slashing is a deterrent, not insurance.
- Resistance to deep L1 reorgs: as with any L2, a deep L1 reorg can roll back L2 state.
- A built-in prover market: the aggregator either brings its own proving capability or procures it.
- **Collusion of the entire builder set plus the aggregator**: under the trust model above this is an out-of-model situation. With no forced inclusion, there is neither a finality bound nor any defense in depth for entry or exit.

2 Roles

- **Builder**: the set of addresses that have registered with the L1 contract and posted a bond. When the lookahead brings a builder's own slot around, it produces a block, signs it, and broadcasts it on the P2P network. The builder is the preconfirmer: the block it signs is itself the preconfirmation commitment for the transactions inside that block.
- **Aggregator**: the service provider that holds the sole seat by way of the perpetual auction. It has exactly one duty: to pack the signature chain into a batch, generate a validity proof, and land it on L1. It does not build, does not order, and holds no discretionary power over content — the only residual discretion is "which prefix to land and when to land it", and both are constrained by deadlines and by permissionless fallback (§6).
- **Fallback lander**: anyone. A fallback lander appears once the aggregator has timed out and is paid under the fault-paid model. This is not a registered role.
- **User**: submits transactions to builders in exchange for preconfirmations. A censored user has no in-protocol remedy.

- **L1 contracts:** the builder registry, the aggregator auction, the candidate entry point (which verifies proofs), and the slashing entry point.
- **The proof system:** the circuit that verifies "this segment of the chain is legal and its state transition is correct". It is the arbiter of this design.

3 Time and scheduling

3.1 Slots and epochs

- **slot = 1 second.** The time unit of L2. There is at most one block per slot, and a block's timestamp is identically the starting second of its slot — the timestamp is not chosen by anyone, it is fixed by the lookahead (which eliminates the entire class of problems around the origin of timestamps found in Appendix C of v15).
- **epoch = 384 slots** (the current value is retained, aligned with L1's 32×12 seconds). In this design an epoch is merely a unit of measure (the granularity of lookahead windows and of fee settlement); it no longer carries the mechanism-level meaning of "one determination per epoch".

3.2 The lookahead

- The lookahead is a mapping $\text{lookahead}(\text{slot}) \rightarrow \text{builder address} \mid \text{VACANT}$. VACANT is an explicit protocol gap, not an error or a special signing key.
- **Computation (one pure consensus function).** A window uses a single finalized registry snapshot and RANDAO seed. From them the protocol computes capped integer quotas and then a seeded placement of the quota tickets. Independent per-slot sampling and capped-weight renormalisation are not alternative implementations; both are deleted.
 - **Window partitioning (if "window" were read as a sliding horizon, the same slot would receive different lookahead assignments at different moments, destroying the determinism of G2):** a window is a fixed aligned partition: $W(\text{slot}) = \text{floor}(\text{slot} / W_{\text{size}})$, $W_{\text{size}} = 384$ (one L2 epoch, and — since an L2 slot is one second and an L1 slot twelve — exactly one L1 epoch of wall-clock time); the starting slot of window W is $W \times W_{\text{size}}$, and "the L1 slot corresponding to that start" is converted by the genesis mapping $L1_slot(s) = \text{GENESIS_L1} + \text{floor}(s / 12)$ (an L2 slot is 1 second, an L1 slot is 12 seconds). The lookahead assignment of any given slot has the same value for every observer at every moment.
 - **A unique snapshot height:** the snapshot height of window W is $H_{\text{snap}}(W) = \text{the L1 slot corresponding to the start of } W - D_{\text{snap}}$, where $D_{\text{snap}} = \text{the lookahead horizon } H_{\text{look}} (2 \text{ L1 epochs}) + \text{the finality distance } F_{\text{final}} (2 \text{ L1 epochs}) + \text{margin} (1 \text{ L1 epoch}) = 5 \text{ L1 epochs}$ (the earlier $D_{\text{snap}} = 3 \text{ epoch}$ omitted the horizon term: the lookahead is required to be computable H_{look} in advance, so the start of the furthest window is $\approx \text{now} + H_{\text{look}}$, and its snapshot point = $\text{now} + H_{\text{look}} - 3 \text{ epoch} = \text{now} - 1 \text{ epoch}$, which is not yet finalized and can be reorged). The setter invariant: $D_{\text{snap}} \geq H_{\text{look}} + F_{\text{final}} + \text{margin}$. It follows that at every moment at which the lookahead is required to be already determined, $\text{randao_source}(W) = H_{\text{snap}}(W) \leq \text{finalized L1 head} < \text{start of } W$ — there is only one seed, it holds simultaneously for all slots in the window, and the lookahead is fixed before it is used and is unaffected by L1

reorgs.

- **The seed:** `keccak256("slot-chain-seed-v1", chainId, protocolVersion, W, randao)` where `randao` is the RANDAO of the L1 epoch containing $H_snap(W)$, authenticated through the EIP-4788 beacon root. Domain separation is normative.
- **The registry snapshot:** the registry state at that same $H_snap(W)$; the entry and exit delay (§4.1) guarantees that the snapshot covers the whole window. The circuit’s lookahead check for every block in the window uses $H_snap(W)$ uniformly, independently of each block’s own anchor block.
- **Quotas:** let $W_MAX_NUM=1$, $W_MAX_DEN=5$, and $Q_MAX=76$. Starting from zero, award each ticket to the unsaturated entry maximizing its bond divided by one plus its current allocation (capped D’Hondt). Compare quotients by integer cross multiplication. Break a tie by the smallest Keccak hash over the domain `quota-tie`, seed, and registration index. Stop after 384 tickets or when every address is saturated; fill the remainder with `VACANT`. Six addresses are needed to fill a window because five can supply only 380 tickets.
- **Placement:** process the 384 positions in order. A real address is eligible when it has a ticket left, differs from the preceding real address, and removing it preserves `remaining[a] <= otherRemaining+1` for every real address. Choose the eligible address with the smallest Keccak hash over the domain `slot-order`, seed, position, and address. `VACANT` may repeat. The result preserves every quota and limits a real builder to one consecutive slot inside a window and two across a boundary. The circuit and clients use this sequence exactly.
- The executable reference implementation is `lookahead-model.py`. Its property tests invoke the production `lookahead()` path itself over adversarial weights, seeds, under-capacity registries and sixty-four-entry registries; a disconnected quota helper is not evidence. The exact algorithm is consensus, while whether the circuit verifies the whole schedule or a schedule commitment is an implementation optimization that must produce the same vector.
- **Lookahead availability.** A window’s schedule becomes computable as soon as its snapshot height is L1-final, that is, as soon as $H_snap(W) \leq L1_head - F_final$. Since $H_snap(W) = L1_slot(W \times W_size) - D_snap$, every window whose start lies at most $12 \times (D_snap - F_final)$ L2 slots ahead of now is already determined. The guaranteed lookahead is therefore

$$12 \times (D_snap - F_final) = 12 \times (160 - 64) = 1152 \text{ L2 slots} \approx 19.2 \text{ minutes}$$

independent of W_size — the window size cancels out of the derivation, because the tightest case is a window boundary falling exactly at the horizon, which can happen for any W_size . $H_look = 768$ slots (12.8 minutes) is the published floor that consumers may rely on; the remaining 384 slots are deliberate slack. Note that the horizon is a consequence of D_snap and F_final alone, not of the window size: shortening or lengthening the window does not move it. The temporal geometry of snapshots, windows and the lookahead horizon:

- **Why "the hash of the most recent proof" is not mixed in as entropy (the design decision; a deliberate divergence here — see Appendix A-1):** one and the same statement can generate an unbounded number of byte-distinct valid proofs (proof generation carries randomness of its own), so a proof hash is entropy that the prover producing it can re-grind at

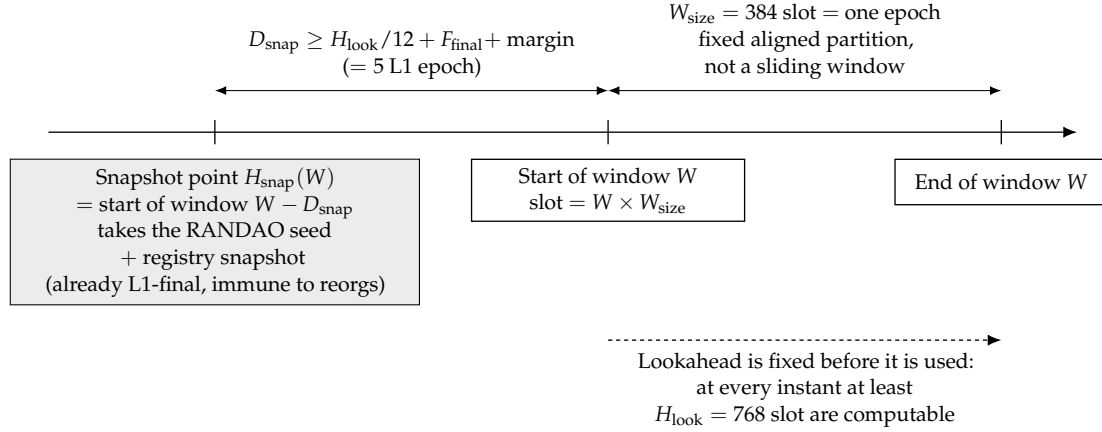


Figure 3: The time geometry of the snapshot point, the lookahead window and the lookahead horizon: entropy and the registry are frozen once and for all at D_{snap} before the window starts, so at every instant at least one full window of lookahead can be computed by anyone.

zero cost — mixing it in would amount to handing part of the control over the lookahead to the aggregator. Pure L1 beacon randomness can be biased by L1 proposers by roughly one bit each, but it is good enough for a use such as the lookahead and introduces no new controlling party. The principle: entropy comes only from the L1 consensus layer, and the moment at which its value is taken is earlier than the lookahead window.

- **A known cost, stated openly:** a public lookahead means that "who produces the block in the next second" is public, so a targeted DoS against a builder about to produce a block is theoretically feasible. Mitigations: a slot is only 1 second long (the attack window is minuscule), builders can use sentry or private networks, and a slot that is DoS'd merely turns into a gap (§5.3). Ethereum itself runs with the same public lookahead.

4 Builders

4.1 Registry

- Maintained by an L1 contract. Registration requires a base nuisance bond plus independently reserved **per-window equivocation tranches** L_{lease} . A schedule window may assign an address tickets only after one previously unencumbered tranche has been reserved for that window; otherwise its quota becomes VACANT. One global, idempotent slash is deleted because it cannot cover both an admissible historical tail and several already-determined future windows.
 - **One tranche prices one window:** $L_{\text{lease}} - R_{\text{evidence}} \geq Q_{\text{MAX}} \times V_{\text{slot,max}} \times \text{safetyFactor}$. $V_{\text{slot,max}}$ is the protocol's published maximum value-at-risk per preconfirmation slot; applications accepting more value receive no protocol guarantee for the excess. Evidence for two headers in slot s slashes the tranche reserved for $\text{windowOf}(s)$. Evidence is idempotent only inside that window, so equivocation in fifteen simultaneously live windows risks fifteen tranches. The historical tail therefore cannot hide behind one already-consumed slash.
 - **The old global-bond arithmetic is rejected:** multiplying an expected 20% share by a selected horizon, or charging only five future windows, omitted sampling variance, align-

ment, detection margin, and the whole still-admissible past tail. Per-window tranches make the liability local and exact: at most $Q_MAX=76$ assigned slots are secured by each independently slashable tranche.

- **Nuisance buffer:** a small amount covering the handling cost of sporadic violations.
- **Honest upper bound:** collateral deters only value visible to the protocol. External bribes and cross-domain positions are unbounded; this is deterrence, not compensation. The net slash after the fixed reporter reimbursement, not the nominal deposit, is the quantity compared with $Q_MAX \times V_slot,max$.
- **Expiry and release are independent of proof progress.** Every state-mutating entry point first advances $min_admissible_slot = \max(old, currentL2Slot - H_slot_expire)$. A candidate containing an earlier slot is invalid. Accepting a normal candidate records, for every schedule window it uses, a bounded `liabilityUntil` at least the candidate window close plus the L1 reorg horizon and evidence margin. A tranche unlocks only after its window ends below $min_admissible_slot$, its `liabilityUntil` has passed, and $\delta_slash + evidenceMargin$ has elapsed. This $O(1)$ -per-window rule terminates during a proving outage and preserves already-admitted candidates. Exiting addresses remain valid in already-snapshotted schedules until expiry or slashing turns their tickets into gaps.
- Registry capacity cap N_max (initially 64) bounds circuit work. At each snapshot the active set is the 64 entries ordered by bond descending, then registration index ascending; replacement requires a bond at least `BUMP_MIN` above the current minimum and takes effect only at a future snapshot. Displaced entries retain all old reservations and liability. Capacity is an engineering bound, not Sybil resistance or strong permissionlessness; one capital holder can occupy it with many keys under the accepted trust model.

4.2 Block production and signing

- The builder whose turn it is at slot s constructs a block header:

```
header = (protocolVersion, chainId, slot, parent_hash, anchor, final_ref,
         fallback_version, fallback_activation_hash,
         txs_root, state_root_claim(optional), coinbase, gas_used,...)
sig = ECDSA-secp256k1-low-s(EIP-712-domain, keccak256(canonicalHeaderEncoding))
```

Key fields:

- **slot:** the slot number of this block; `timestamp` is uniquely determined by it and does not exist separately.
- **anchor:** the L1 block referenced by this block (the freshness baseline and the $L1 \rightarrow L2$ consumption point, §7; `m_consumed` in §7 is determined by the block header's anchor, so it must be a block-header field covered by the signature rather than a batch-level implicit quantity).
- **parent_hash:** the hash of the parent block header. Having the signature cover `parent_hash` is a central element of this design: the choice of parent (including "whom to skip") is an explicit act that the builder has signed, not a fuzzy state that can be repudiated afterwards. The link uses the hash of the parent block header rather than the parent's signature — the hash is structurally necessary (it defines the chain), whereas putting the parent's signature into the child block as well is redundant (the circuit verifies every block's signature anyway); it may serve as an evidence-packaging convention at the P2P layer, but it is not

a consensus rule.

- **Signing-domain uniqueness invariant:** version 1 uses ECDSA over secp256k1, Keccak-256, low-s signatures and normalized v. The EIP-712 domain contains `chainId`, the verifying contract and `protocolVersion`; the struct hash covers every header field in the listed order. The L1 contract and circuit verify the same canonical bytes and reject every non-canonical signature encoding. A future BLS or SNARK-friendly scheme requires a new protocol version and simultaneous verifier upgrade; it is not an implementation choice.
- The builder broadcasts (block header + signature + block body) over P2P. This signed block is itself the preconfirmation: the receipt given to the user = (block header, builder's signature, inclusion proof of the transaction) — the receipt contains the block header hash and `txs_root`, and is publicly forwardable evidence (used in §5.5).
- The decision procedure of the two-tier parent rule (the main statement and its exceptions are given in the next item):

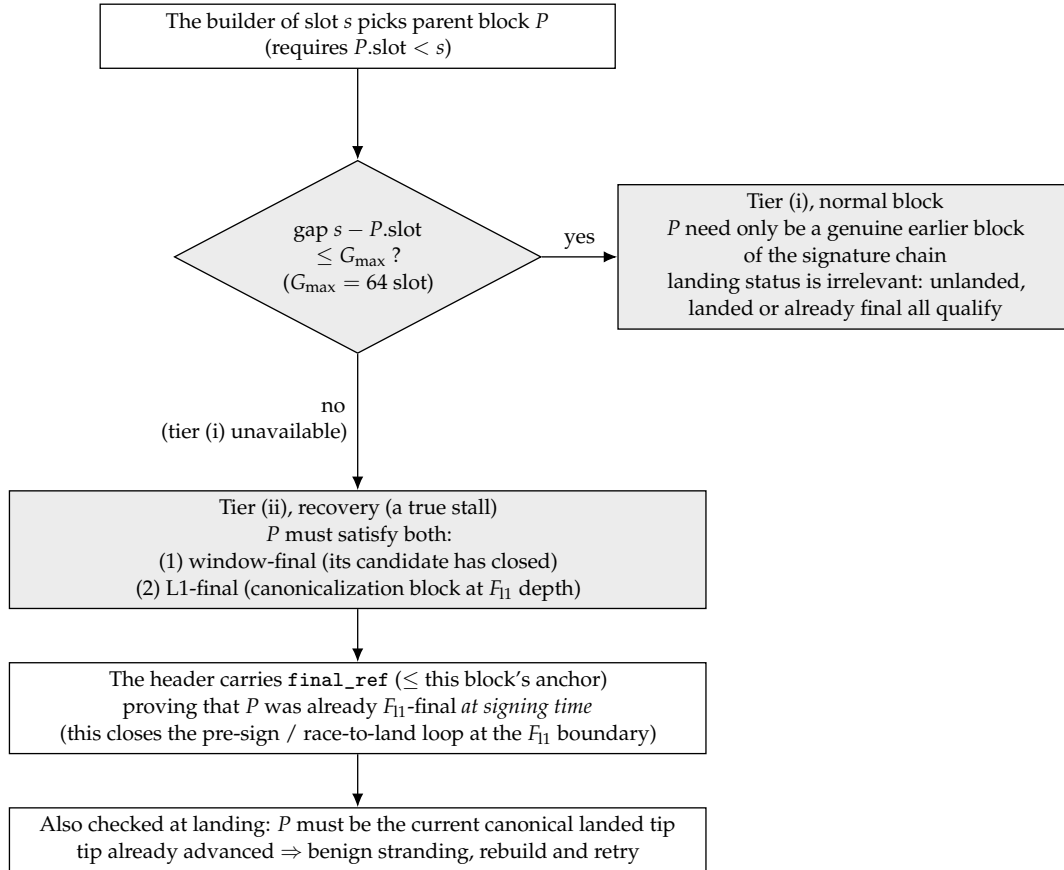


Figure 4: The decision procedure of the two-tier parent rule. The criterion is the size of the gap on the signature chain, not the landing status of the parent, so the rule evaluates identically at signing time, at proving time and at landing time; only a true stall, whose gap exceeds $G_{\max}$, drops into the recovery tier that demands a final parent.

- **The parent rule — two tiers, with the criterion being [the size of the gap] rather than the parent's landing status:** `parent.slot < s`, and the parent satisfies one of the following. The criterion is the gap $s - \text{parent.slot}$, evaluated over the structure of the signature

chain (the gap + parent_hash) — the evaluation at signing time, at proving time and at landing time agree, and it does not depend on whether the parent "has landed / is already final" at this instant, a state that drifts with time:

- **(i) Bounded-gap tier (normal block production):** $s - \text{parent.slot} \leq G_{\text{max}}$ (the gap cap, initially 64 slot). The parent need only be a real earlier block in the signature chain (parent_hash points at it), and its landing status is immaterial — not landed, landed but not final, or even already final are all admissible. Normal block production (including the next block after every landing) always takes this tier: after a landing, the next block attaches with a small gap to the (landed) chain head, and a gap $\leq G_{\text{max}}$ is legal, so "landed but not final" therefore constitutes no coverage hole at all (the criterion is the gap, not the landing status, so there is no class of parent for which "neither tier applies"). This tier guards against deep reorgs: G_{max} caps the rollback depth of a single block, and a malicious builder that wants to orphan a deeper unlanded tail needs a relayed sparse branch (see the residual risk below).
- **(ii) Unbounded-gap tier (recovery):** $s - \text{parent.slot}$ is arbitrary, but the parent must be the canonical head and the L1 block that made it canonical (a normal close or fallback commit) must have reached F_{11} depth. Depth is never measured from an earlier provisional candidate transaction. A header field finalRef authenticates that canonicalization block at signing time. During fallback the additional episode binding of §6.3 applies. This tier is entered only when the gap exceeds G_{max} ; its unbounded gap permits recovery after a stall of any duration without treating a shallow canonical decision as final.
- A tier-(ii) header carries finalRef, an authenticated L1 reference proving that the parent's canonicalizationBlock had already reached F_{11} depth when the builder signed. The reference must not post-date the block's anchor. This prevents pre-signing against an unfinalized head and racing the header into the first block at which that head merely becomes final.
- **Justification for the value $G_{\text{max_landed}} = \infty$ (the two-tier parent rule):** tier (ii) has no cap because the reorg exposure through a landed head is naturally bounded by the length of the unlanded tail, and the tail length is bounded by Δ_{lag} — a tail older than Δ_{lag} has already been landed by the fallback, is final, and can no longer be reorged. "Unbounded" therefore does not increase the worst-case reorg depth (still $\leq \Delta_{\text{lag}}$, the same bound as the existing residual risk in §5.4), yet it lets a stall of arbitrary duration be recovered in a single hop (see §8). A finite value loses at both ends: below the proving latency it can never catch up with a long stall, and set equal to Δ_{lag} the tightening is an illusion (nature already gives Δ_{lag}) —
- **Conditional tail bound:** without continuous protocol-level DA, the maximum rollback is the actual unlanded tail. It stays near the final-lag threshold only when a funded prover and an uncensored L1 transaction complete fallback on time; otherwise it grows with the stall. Normal settlement prefers the heaviest proven tail. Fallback intentionally does not: the first qualifying proof may discard the entire unfinalized tail. Tier (ii) therefore lowers the builder-side cost of proposing that rollback to one scheduled recovery block, and users must treat every omitted tier-2 promise as revoked.

4.3 Equivocation slashing (verified directly on L1)

- **Definition of the fault:** the same builder signs two byte-wise different block headers for the same slot.
- **Evidence and enforcement:** anyone may submit two header/signature pairs. The L1 contract verifies the canonical ECDSA signatures, equal slot, unequal header hashes, matching scheduled signer and an unslashed tranche for `windowOf(slot)`. It burns that `L_lease` tranche in full. The submitter receives only the fixed `R_evidence` reimbursement from the protocol fee pool; it is never carved out of the tranche. A self-submitter therefore cannot reduce the net slash below the value used in the collateral invariant.
- **The point at which a slashing takes effect is a deterministic, slot-based rule :** if the timestamp of the L1 block that includes the slashing transaction corresponds to L2 slot `s_0`, then the slashing takes effect from slot `s_0 +  $\delta_{\text{slash}}$` (initially $\delta_{\text{slash}} = 64$ slot): once it is in effect, every slot already scheduled to that builder in the lookahead is treated as a gap — the circuit rule: when a batch lands, the L1 contract hands the list of (slashed party, effective slot) entries it has itself recorded to the proof verification as a verified input; any block within the batch that is "signed by a party whose slashing is already in effect, with slot $\geq$ the effective slot" is invalid. Effectiveness is delimited by slot (rather than by each block's anchor block), which guarantees that all forks agree on the judgement of "who has already been slashed", so that no anchoring race exists. Without this clause, a slashed builder with a zero bond could still produce legal blocks (§4.1 and §5.1 would then contradict each other).
- **Effectiveness is slot-based and expiry is a global watermark:** a block signed by a slashed builder is invalid when `header.slot  $\geq$  effectiveSlot`; every block is invalid when `header.slot < minAdmissibleSlot`, except that a candidate already accepted into a still-open normal window remains governed by that window's frozen admission version. Neither rule depends on the later landing time. The forward rule converts the builder's future tickets to gaps; the monotone watermark makes historical liability expire even when no proof can close a window.
- **Rejected retroactive rules:** keying invalidity on evidence landing time lets the equivocator invalidate an already-built honest tail by timing its own evidence. Deferring invalidity to a later settlement window instead adds a full-window exposure period. The adopted rule is immediate only for future slots and never retroactive; the independent historical watermark bounds old liability.
- **Historical normal-mode blocks:** a pre-effective, non-expired block remains admissible in normal settlement and its window tranche remains slashable. It does not qualify as fallback recovery: the first fallback block has a slot at least equal to the episode activation slot and binds `fallbackVersion` and `fallbackActivationId`. Thus a zero-bond key cannot backdate a first-valid fallback chain. Equivocations in distinct schedule windows slash distinct tranches.

5 The signature chain: validity, canonicity, preconfirmation semantics

5.1 Single-block validity (circuit rules)

A block is valid if and only if all of the following hold (all of them circuit-verifiable):

1. The signature is valid and the signer = lookahead(slot) (the lookahead is recomputed inside the circuit from L1-anchored data). There is no exception to this rule (after This design does not include forced inclusion, the P_forced forced-only-block exemption went with it — the right to produce a block belongs to the scheduled builder alone; the cost is that under a full cartel there is no longer an any-key escape path for block production, see §8 and §1). This replaces the old "three conditions" formulation that this design does not use;
2. parent.slot < slot, and the parent block satisfies one of the two tiers of the §4.2 rule (the criterion is the size of the gap, not the parent's landing status): (i) when the gap satisfies $\text{slot} - \text{parent.slot} \leq G_{\text{max}}$, the parent may be any real earlier block in the signature chain (landing status is irrelevant); (ii) the gap has no upper bound, but the parent must be an L1-final landed block. parent_hash points to that valid parent. A tier (ii) block additionally requires a valid final_ref: final_ref witnesses that the parent has reached F_l1-final and that final_ref ≤ this block's anchor (§4.2); that the parent "is the current canonical tip" is still checked by §5.6 at landing time (every candidate is verified against the frozen window baseline; the two predicates are kept separate: at signing time finality is proved by final_ref, at landing time canonicity is verified by the entry point);
3. slot does not run past the range declared by the batch that lands it; the timestamp rule is automatically satisfied (= slot);
4. The execution of the transactions in the block is valid (ordinary state-transition verification); the anchor tx satisfies the freshness rule (§7) and satisfies the §7 causal-ordering invariant $\text{anchor.L1_timestamp} \leq \text{L2 timestamp(slot)}$; and inbound L1→L2 messages are consumed per §7 up to the [unified maximum prefix] — the longest prefix of the FIFO order that simultaneously satisfies "count ≤ C_anchor" and "cumulative declared gas ≤ the L1→L2 message gas share"; taking that longest prefix is valid, and only taking less than it is invalid (this rules out the indefinite censorship of "using a fresh anchor while processing zero inbound messages").

5.2 Canonicity and fork choice

- **The portion already landed on L1:** the canonical chain = the chain obtained by stringing together the window-final batches at the close of each settlement window (§5.6). Conflicting candidates within a window are superseded by the §5.2 total order, and the close settles the matter for life; only after the close is a candidate "permanently out".
- **The coordination convention for the unlanded portion:** honest nodes use the total order below. It constrains P2P convergence, not malicious behavior. In NORMAL mode the settlement window mechanically lets a heavier proven candidate supersede a worse one. In FALLBACK_OPEN, that protection is intentionally suspended: the first qualifying episode-bound proof commits immediately under §6.3.
- **The best-chain total order — shared by three places: fork choice, landing strategy, and the §5.6 window candidate comparison.** Given a fork point F, the candidate chains extending F are compared, in the following order: **The object of comparison is a scalar triple, not a pairwise structural comparison.** (The old "look at the first differing block of the two chains to determine the lane" was a pairwise criterion, and a non-transitive cycle $A > B > C > A$ can be constructed: $A=[X,a]$ vs $B=[X,b1..b3]$ diverge at $a/b1$, so content wins; B vs $C=[Y,c1,c2]$ diverge at X/Y , so block count decides; C vs A again diverge at Y/X , so

block count decides; a cycle. Fix: the quantities compared must be each candidate's own invariant scalars, relative to the fixed baseline F and independent of the object of comparison). Each candidate chain extending F independently computes the triple $\text{key} = (\text{count}, \text{tip_slot}, \text{tip_hash})$:

1. **count**: the number of valid blocks starting from F ;
 2. **tip_slot**: the slot of the tip;
 3. **tip_hash**: smaller wins after equal count and tip slot. Such a difference implies equivocation by the scheduled builder. The comparison is the lexicographic order (count descending, tip slot descending, tip hash ascending), and applies only to P2P choice and normal settlement. Fallback does not use this order after activation: a frozen or heavier unfinalized tail may be discarded by the first qualifying fallback commit.
- The final arbiter is still L1: disagreements are resolved once and for all by the §5.6 total order at the close of the settlement window. Equivocators are slashed (§4.3); blocks that forked innocently because of a network partition become orphans, and the transactions in them return to the mempool to be repacked.
 - **Normal-mode landing strategy**: a lander applies the total order to its view; a heavier proof can correct a worse candidate before normal close. This is an economic strategy, not an adjudicated duty. Once fallback activates, the strategy no longer produces a protocol guarantee: the first qualifying episode-bound proof wins, even if another observer knows a heavier tail.
 - **The visibility assumption and the "economic only, not enforced" characterization**: this design assumes that, under normal P2P propagation, the aggregator (and any lander) can in most cases see the currently longest available signature chain — but the protocol cannot, and does not attempt to, force a lander to land the longest chain. The reason is structural: visibility of the chain can itself be taken away — the simplest example is a builder at the tail of the chain withholding the block it has just signed (§5.5 withhold-body / withhold-signature); the aggregator cannot obtain it and therefore cannot include it, and that is not a fault. Hence "land the longest visible chain" is a rational choice under economic incentives, not a slashable obligation: a lander that lands a shorter chain is not penalized, and the consequences run through only two economic channels — (i) being superseded by a heavier candidate inside the §5.6 window, having paid gas and proving fees for nothing; (ii) under the base-fee revenue-sharing rule below (§5.6/§6.5), the shorter the chain landed, the smaller the base on which the share is computed. the earlier attempt to turn "ought to have landed but did not" into an adjudicable fault, and the refutation of that attempt, is precisely the cautionary example for this characterization.
 - **A node's local chain-selection rule = the total order above**: the criterion an L2 node uses to dynamically maintain its "local best signature chain" is exactly the triple total order above; no second set of rules is needed. Equivocation is heavily slashed under §4.3, but one cannot assume that it does not happen — when it does, two signed blocks appear at the same slot and the chain splits in two; the two forks are still two individually valid candidate chains, and the triple yields a total order as usual (differing block counts are settled by count, equal block counts are arbitrated by $\text{tip_slot}/\text{tip_hash}$), so the convergence of node views is not interrupted by equivocation; slashing the equivocator (§4.3) is orthogonal to chain selection. Why the fee total does not enter the total order (a design trade-off): it was at one point considered to make "the sum of all base fees on the chain" one of the

ordering inputs. The conclusion is that it cannot enter the canonicity criterion: fees are a quantity an attacker can manufacture by paying itself — a malicious builder that stuffs the block in its own slot with high-fee self-transfers can make a shorter chain beat an honest longer chain on the fee dimension, which amounts to buying out the honest majority's chain-selection outcome with capital (and if part of the fees flows back to the lander, the colluder's cost of manufacturing those fees is even partly refunded). The total order therefore keeps the number of genuinely signed blocks as its primary key (forging one requires stealing a builder's private key and cannot be manufactured); the fee total enters only the reward (the §6.5 base-fee sharing) and not the ordering — incentive alignment relies on money, canonicity determination relies on signatures.

- **Honest bounding:** before landing, an equivocating builder + one colluder willing to land its fork (the aggregator, or any lander inside the fallback window, or even a bribed L1 proposer) can make the "other" tail canonical and isolate the preconfirmations contained in it. This attack (i) requires one equivocation that L1 can slash directly (the cost = L_{eq} , priced by the exposure accumulated over the window, §4.1); (ii) has a depth bound = the length of the unlanded tail. The tail length is not "hard" capped by Δ_{lag} : $lag > \Delta_{lag}$ only opens the authorization for fallback landing, it does not force any batch to be accepted; while an aggregator that withholds landing drags things out, the tail keeps growing throughout the fallback response time. The honest depth bound = Δ_{lag} + the fallback response time (one round of proving
 – landing $\approx 10\text{--}15$ min $\approx 2\text{--}3$ epochs), and it can be stretched further by sustained L1-level censorship of the fallback transaction — the latter being the most expensive censorship class, already priced in §6.3. This is a soft upper bound plus an explicitly stated response-time assumption, not a hard cap. Therefore the safety of a preconfirmation before landing is deterrence plus bounded depth, not absolute — the user-facing semantics of §5.4 are worded accordingly. Keeping the tail short (§6.3) is the principal means of shrinking the payoff of this class of attack.
- **Canonicity is still determined by L1 alone (G2) — but L1 does not adjudicate "which chain ought to be landed"; the settlement window selects it mechanically:** off-chain fork choice gives honest nodes convergence (the total order above), while "the best chain wins" is delivered mechanically by the §5.6 settlement window — L1 does not adjudicate "who ought to land what", it only lets the heaviest proven candidate inside the window become final. This design is therefore not "L1 enforces the longest chain" (that would require protocol-level DA in order to see the unlanded tail) but "L1 picks the heaviest among the already-landed, already-proven candidates" — no DA, no ex post adjudication; it is the mechanical middle point chosen under the §1 constraints.

5.3 Gaps

- No valid block appears at slot s (the builder went offline, was DoSed, or its block body did not propagate), the chain is left empty at s , and the builder at $s+1$ builds on an earlier chain head. Gaps are handled as a normal case, not as an anomaly: no proof of "why it is missing" is required, and nobody is slashed for a gap (producing a block is an opportunity, not an obligation — inheriting the principle of v15 §9.3, but now applying to every slot).
- The cost of a gap is absorbed by incentives: a skipped slot earns no fees, and users who hold preconfirmations settle accounts with that builder's reputation. Nuisance-level chronic

absence is handled by the registry's liveness rules (an optional item, §11).

5.4 Deletion vs. gaps: what the signature chain fixes and what remains (stated explicitly)

The parent-block-hash chaining fix raised by the design has precisely the following effect:

- **Fixed: the aggregator's selective deletion.** Because a child block's signature covers parent_hash, pulling a block out of the middle of the chain breaks the chain for every successor and the proof necessarily fails. Over a signature chain, the aggregator retains only the power to take a prefix: land $[. . m]$ and leave $[m+1 . . k]$ outside the chain. (The deep-reorg path in which "a malicious builder starts a short alternative branch from an old point without equivocating, for the aggregator to fork-choose"; a single block is capped by $G_{\max}$, but a relayed sparse branch can reach the entire unlanded tail) And the suffix that is left behind still attaches to the new L1 chain head, so the next batch (anyone's) can land it just the same — truncation is delay, not destruction. Together with the landing deadline (§6.3), a malicious truncation by the aggregator buys little.
- **What remains: the skip by an adjacent builder — and it is profitable.** Builder $s+1$ builds on $s-1$ and claims never to have seen the block at s — which cannot be falsified inside the protocol ("I did not receive it" is unprovable). previously extrapolated "the actor gains nothing" onto skipping, which was wrong: the skipper can repack the high-priority-fee transactions and the MEV from the skipped block s into its own block and still collect the entire revenue of its own slot — a skip is a profitable, non-slashable revocation of preconfirmations that a single untrusted builder can carry out on its own. Four points of honest bounding: (1) the choice is fixed by a signature and publicly visible (parent_hash in black and white, plus the signed block at s circulating over P2P), so attribution has hard evidence; (2) the one-shot payoff of an adjacent skip is bounded by the fee/MEV transfer of one slot; **Deep skip / sparse private branch:** with a single block, one malicious builder can, in a single deep skip, affect at most $G_{\max} - 1$ unlanded blocks; but a minority coalition that relays several blocks along a private branch can affect the entire unlanded honest tail. The coordination convention as revised in §5.2 makes the honest network not follow such a fork, so it must be landed on L1 ahead of the others in order to take effect. The crux of the race: if the colluding lander is only a fallback lander, this really is a race against the honest batch already in flight (the window $\approx$ the normal landing cadence, and the fallback right does not open at all before $1\text{ag} > \Delta_{1\text{ag}}$); if the colluder is the incumbent aggregator, then during normal operation it is the only routine lander and there is no race to speak of. This combination is a security residual risk that this design prices explicitly, and its depth bound splits into three tiers according to the form of the attack:
 - **Tier (i), single-block deep skip (unlanded parent):** $\text{depth} \leq G_{\max} - 1$ (a tail of roughly 1 minute), one malicious slot;
 - **Tier (i), relayed sparse branch:** $\text{depth} \leq$ the unlanded tail $\approx \Delta_{1\text{ag}}$ + the fallback response, at the cost that the coalition must hold one scheduled slot in every $G_{\max}$ window of the tail (the density requirement becomes harsher as $G_{\max}$ is tuned smaller);
 - **Tier (ii), single block via a landed head:** one malicious builder produces a block whose parent is a final landed head, and in a single step can supply a branch that isolates up to the entire unlanded tail, with no coalition density required. **Precondition:** for that branch to be landable on L1 by a colluding lander, its final parent block must at the

same time be the current canonical landed tip (§5.6 only lets candidates that extend the current tip land) — that is, the current tip must itself already be final. In steady state the tip is usually already final, so the precondition usually holds and the threat is not weakened; the point here is only to state the residual risk accurately (when the tip is not final, this single-block attack must first wait for the tip to become final, which has the same origin as the F_{L1} wait for recovery in §4.2). **The depth upper bound $\leq$ the length of the unlanded tail**, which lowers the builder-side cost of a fully isolating branch from "a dense coalition" to "one person, one block"; the worst-case depth and the binding constraint that "a lander must collude" both stay unchanged — this is the price accepted for $G_{\max_landed} = \infty$, in exchange for one-hop recovery from a stall of arbitrary length (§6.4/§8). " $\approx \Delta_{lag}$ " is a conditional upper bound: the tail length is $\approx \Delta_{lag}$ only if permissionless fallback lands on schedule (the same condition as the §6.3 liveness bound under a Byzantine aggregator); when fallback stalls, or the landing transaction is censored by L1, the tail grows without bound along with the stall, and a single tier-(ii) block can isolate a tail far longer than Δ_{lag} — see that conditional argument in §4.2 for details. This is not unique to ∞ (a tier-(i) relay coalition can likewise isolate an over-long tail, merely at greater cost), but ∞ lowers the cost in that regime to one person and one block, and this must be stated honestly alongside it. The frequency of both tiers is $\leq$ the number of scheduled slots the coalition obtains, everything is attributable throughout (the parent choice is signed and on record, and the aggregator's truncated landing is public on L1), the only mechanical deterrents are seat value and reputation, and the protocol does not slash builders; but, for these isolating branches actually to be finalized they must in addition beat a heavier honest candidate inside the §5.6 settlement window — as soon as somebody lands the honest tail together with its proof into the window, the isolating branch is self-defeating; the residual risk narrows to "nobody lands the honest tail inside the window" (the fallback feasibility condition, §6.3). Complete structural elimination still requires protocol-level DA (Appendix A-3), which is not adopted, per §1. $G_{\max}$ is thereby a governance knob: tuning it up strengthens gap tolerance (§6.4), tuning it down raises the coalition's scheduling density required for a relayed sparse branch. The direct repacking payoff is still bounded by the capacity of the attacking block itself; the frequency of both kinds of skip is bounded by the fraction of dishonest builders — the "honest majority" of the §1 trust model is load-bearing exactly here (this is not conservative redundancy); (3) the harm to users is layered: pure "inclusion"-class commitments are usually honored in the next slot via repacking (the skipper has exactly the incentive to collect the fees of those transactions), while what is genuinely broken are the "ordering / exact state"-class commitments; (4) the handling = public attribution + the registry's liveness/reputation rules (item 1 of §11) + users penalizing repeat offenders with their flow. This is a deterrence-grade residual risk, which this design accepts and states explicitly; structural elimination requires protocol-level DA / availability proofs (for the cost identity see Appendix A-3).

- **Corollary — preconfirmation semantics:** there are four ways in which the preconfirmation of a signed block can be broken:
 1. The builder that signed it equivocates and a colluding party lands the other fork — the cost is being slashed directly by L1 for L_{eq} , priced by the exposure accumulated over the window (§4.1); the isolable depth is $\leq \Delta_{lag}$ + the fallback response time (see the honest bounding in §5.2);

2. Some later builder skips it — profitable for the skipper (fee/MEV transfer) and not slashable; publicly visible and fixed by a signature; an adjacent skip affects a single slot, a single-block deep skip affects at most $G_{\max} - 1$ unlanded blocks, and a minority coalition's relayed sparse branch can reach the entire unlanded tail; it generally has to win the landing race— but no race is needed when the incumbent aggregator colludes; the frequency is bounded by the honest-majority assumption; pure inclusion-class commitments are usually honored via repacking (§5.4 above);
3. The builder that signed it withholds the block body (§5.5) — the block can never be landed, the preconfirmation lapses as a matter of course, and the handling is likewise deterrence-grade;
4. **A deep L1 reorg** is a residual risk shared by every L2. Product surfaces must distinguish inclusion intent, revocable ordering/state preconfirmation, canonical L1 settlement, and L1 finality. Tier-2 promises are additionally revoked by fallback activation: blocks omitted from the first qualifying episode-bound chain are dropped and their transactions return to the mempool.

5.5 Signing the header without publishing the body (withhold-body) — a known non-slashable misbehavior

- **The behavior:** the builder broadcasts (block header + signature) but withholds the block body. The consequences: later builders cannot compute its state and can only skip it; the aggregator cannot obtain the data and can never land it; the preconfirmations of users holding receipts lapse.
- **Structural self-healing:** a builder can only build on a parent block whose body it holds and whose state it can execute out — this is a physical fact of execution, and need not be "required" as a rule. A suffix whose bodies are withheld will therefore be automatically bypassed as soon as the first non-colluding builder's turn comes (it has no choice but to attach to the last available block), producing an available fork that anyone can land. For a withheld-body chain to hold up the finality of the whole chain, every scheduled builder after it would have to keep colluding to share the bodies privately — that is the full-cartel case. Since This design does not include forced inclusion there is no backstop clause left in the protocol for it; see §8.
- **The characterization, stated explicitly:** this is not a slashable fault — "it has the body but does not publish it" is unprovable inside the protocol (the same undecidability as "skipping" has at its root). This design does not pretend to solve it; the handling is:
 1. **Preconfirmations before landing explicitly carry a DA assumption:** user documentation must state that "a preconfirmation before landing relies on best-effort availability of the block body on the P2P network, with no protocol-level DA guarantee";
 2. The receipt format (§4.2) leaves the victim user holding (block header + signature + inclusion proof) — public evidence of the withholding behavior, for use by the reputation system and by off-chain accountability;
 3. The withholder itself gains nothing (its block earns no fees and is bound to be skipped), so this is purely spiteful behavior, and deterrence rests on reputation and the registry's liveness rules (item 1 of §11).
- The slashable faults are mechanically adjudicable: per-window builder equivocation and

objective aggregator SLA breach. Skipping and body withholding remain non-slashable. A worse normal candidate can be superseded; a qualifying fallback candidate cannot. Fallback activation explicitly expires omitted tier-2 promises.

5.6 Settlement windows and heaviest-proven-batch finality (settlement-window finality) — mechanical, DA-free tail protection

The window lifecycle in a single diagram (in one-to-one correspondence with the pseudocode below and with the executable model of Appendix C):

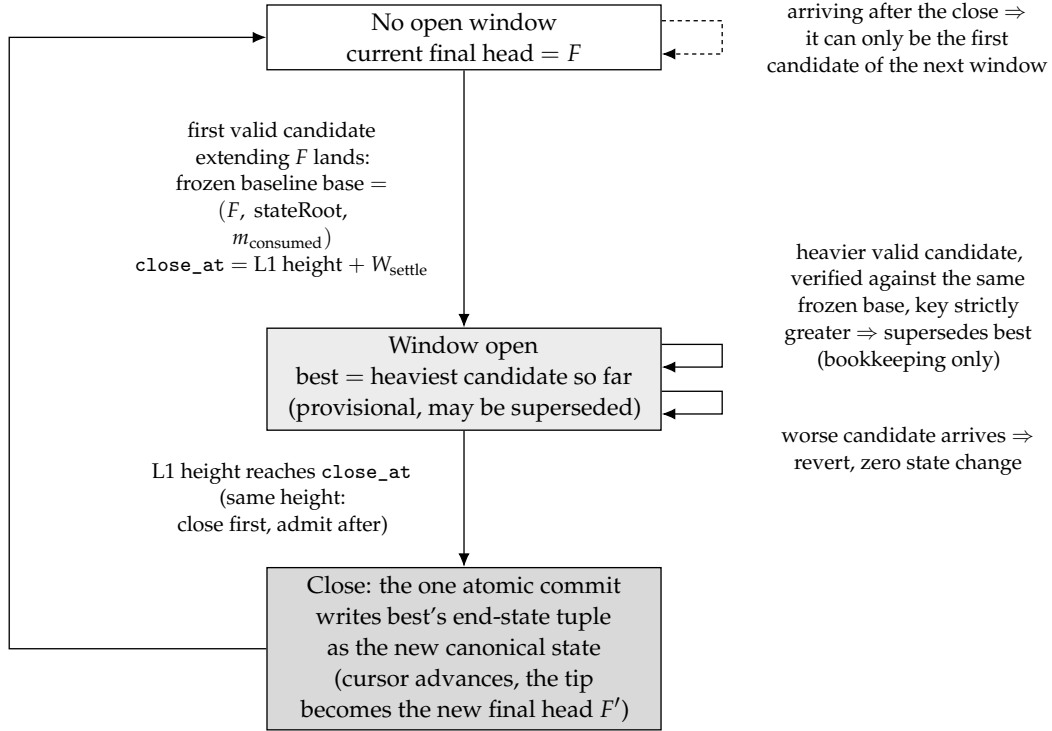


Figure 5: The life cycle of a settlement window as a state machine: opening the window freezes the baseline, every candidate inside the window is verified against that same baseline and only updates bookkeeping, and not until the close is the heaviest candidate's end-state tuple committed atomically as the new canonical state.

- **Candidates and the settlement window — including the [baseline freeze + candidate versioning + close commit] state machine (a restoration of "the first to land wins permanently"):** let the current window-final head be F . Opening the window immediately freezes the baseline tuple $\text{base} = (F, F.\text{stateRoot}, m_{\text{consumed}})$; every candidate inside the window is verified against the same frozen base (the proof's starting cursor/state root = base, consuming the same queue head), and passing verification records only that candidate's own end tuple $\text{end}_i = (\text{tip}_i, \text{stateRoot}_i, m_{\text{consumed}_i})$ together with its §5.2 weight key_i — provisional acceptance changes no canonical state (it does not advance cursors, does not switch parents, and does not execute bridge effects). At window close, only the end tuple of the current heaviest candidate (the one with the largest key) is committed atomically, in one step, as the new canonical state; its tip becomes F' , and the next window starts from the new base derived from F' . Pseudocode:

```

openWindow(F): base  $\leftarrow$  (F, F.stateRoot, m_consumed); best  $\leftarrow$   $\perp$ ; close_at  $\leftarrow$  L1.now + W_settle
acceptCandidate(c, expectedVersion): sync(); require(mode!=FALLBACK_OPEN);
require(expectedVersion=version); requires best= $\perp$   $\vee$  L1.now < close_at;
# at the same L1 height, closeWindow is evaluated first, acceptance afterwards;
# a candidate arriving after that point can only open the next window
if best= $\perp$ : openWindow(current window-final head); # the first candidate opens the window
verify(c.proof, from = base); # all candidates share one frozen baseline
requires best =  $\perp$   $\vee$  key(c) > key(best); # §5.2 triple lexicographic order, strictly heavier
best  $\leftarrow$  (c.end, key(c)) # bookkeeping only, canonical untouched
closeWindow: requires L1.now  $\geq$  close_at  $\wedge$  best  $\neq$   $\perp$ ;
canonical  $\leftarrow$  best.end; # the single atomic commit
credit(best.beneficiary,
 $\varphi_{\text{land}} \times \Sigma \text{base\_fee}(\text{best})$ ) # the base-fee share is booked at close (§6.5)

```

(What the baseline freeze covers is [the cursors and the state]; it does not include queue contents: the L1→L2 message queue on L1 is append-only, entries are referenced by global sequence number, and an entry's content is immutable once enqueued — so there is no need to "freeze a queue snapshot": two candidates starting from the same frozen cursor necessarily read the same content at every sequence position; new entries enqueued midway through the window are visible and determinate to any later candidate "whose blocks can reach them" (a heavier candidate legitimately consumes more), and the window state remains a pure function of L1 history — enqueue events included. In implementation this requires the queue commitment to be organized per-entry and append-only (a hash chain or an MMR), and proofs to reference entries by sequence number; a "single mutable queue root as of the moment of landing" must not be used as a public input — otherwise the public inputs of successive candidates within a window would drift. Model property P12. `closeWindow` may be called by anyone, or folded into the first transaction of the next window as a lazy close; the adjudication order at one and the same L1 height is normalized to "close precedes acceptance" — candidates arriving at or after the `close_at` height do not take part in this window and can only be the first candidate of the next window, and any implementation (including lazy close) must be consistent with that semantics (the replay of the executable model is exactly this semantics, Appendix C P5b/P6); the window state and best are both pure functions of L1 history, and under a shallow L1 reorg they roll back together with L1 — folded into item 9 of §11.) Stated honestly: this is a small window state machine (three actions: baseline freeze / candidate versioning / close commit) — far smaller than challenge/deposit/DA or the old episode, but It is a small state machine, and this document says so rather than claiming otherwise. The atomic advance rule for the `m_consumed` cursor of §7 is correspondingly re-read: both the starting check and the advance are relative to the window baseline and the close commit, and no longer to "whoever lands first advances".

- **Why "the heaviest proven batch" is by itself a mechanical protection of the tail:** an honest lander brings the complete honest tail (necessarily the heaviest under the §5.2 total order — under an honest majority, any branch that excludes honest blocks contains fewer blocks, see §5.4) onto L1 together with its proof, and it wins outright. What if a malicious lander gets ahead by landing a worse chain? Inside the window it is overwritten by the honest candidate; nothing is left on chain and no compensation has to be adjudicated, and the malicious party has paid L1 gas plus proving fees for nothing (a natural griefing cost that requires no design). Every "how do we prove that it should have landed at the time" difficulty of the earlier accountability-based attempts ceases to exist: the chain that ought to land lands by itself and wins, and nobody needs to prove "that it once existed".
- **Normal-window sniping:** a last-moment heavier normal candidate contains more genuine signed blocks and may supersede the earlier best before close. This statement does

not apply after fallback activation, where first qualifying acceptance is final and losing provers may bear duplicate work.

- **The setter invariant for W_{settle} :** $W_{\text{settle}} \geq P_{\text{prove,max}} + T_{\text{include,max}} + \text{margin}$ — this guarantees that, from window opening onward, anyone who sees a heavier tail has time to prove it and land it. An honest lander need not wait for the window: it starts proving as soon as it sees the tail, so under normal conditions the heaviest candidate is already on its way when the window opens. The steady-state cost is zero: with no competition the window contains only the aggregator's own single candidate, and nobody spends an extra cent — the competition mechanism is exercised only when the system is under attack.
- **The window state is a pure function of L1 history:** candidates are all L1 transactions, window opening and close are L1 heights, and the current best is the deterministic result of transaction-by-transaction comparison — any node that replays L1 obtains the same window state; under a shallow L1 reorg it rolls back together with L1 (folded into the atomic-rollback list of item 9 of §11, with no extra state machine).
- **Liveness and anti-spam :** superseding requires a strictly greater weight plus a valid proof, so a candidate spammer must pay a real proving fee every time and is capped by the length of the real tail — there is no livelock; a candidate arriving after window close is benignly stranded (it targets the old F , and it suffices to rebuild it onto the new F' , the same stranding semantics as in §6.4). The candidate's beneficiary (the reward/refund address) is built into the proof's public inputs, so mempool copying cannot change where it points.
- **Explicitly stated residual risks (stated honestly):** (a) temporary ordering swings during the window: the "current best candidate" inside a window may be superseded by a heavier one, so the temporary head that L2 nodes follow is mutable within the window — this corresponds precisely to the existing tier-2 preconfirmation semantics (revocable, decided by window close); it adds no new exposure and merely makes that exposure explicit; withdrawals and bridging execute only from window-final state (§6.1/§6.2). (b) **Fallback for excluded blocks:** if the heaviest candidate still excludes certain genuine blocks (for example, an attacker wins some window with a sparse branch built on enough slots of its own in the lookahead — under an honest majority this requires a coalition with dense lookahead scheduling, bounded as in §5.4), the excluded transactions return to the mempool as before and are repacked in the next window — the worst-case depth is still $\leq$ the unlanded tail, the same bound as the existing residual risk of §5.4, and C does not widen it. (c) **Finality + W_{settle} :** this is the real cost of the settlement-window design, and this is accepted explicitly (accounted for in §6.2).

6 Landing

6.1 Batches

- A batch = a contiguous segment of the signature chain $[m+1 \dots k]$ (extending from the current window-final head m , §5.6) + references to the block-body data published as blobs (next bullet) + a validity proof that verifies all the rules of §5.1 together with the per-block state transitions.
- **Atomic landing + window finality :** a candidate is still created by one transaction — data references, proof and state root are submitted together and verified on the spot by L1, and publishing data creates no claim on anything, so there is still no "proposed but not

yet proven" intermediate state to reason about — there is still no "proposed but not yet proven" intermediate state, no sealing deadline / cancellation horizon / cancellation cascade / timing-exemption net (the v15 complexity reduction is retained). But what "verification passed" now yields is the current best candidate (provisional) of that window, rather than immediate finality; finality is granted at settlement-window close to the heaviest proven candidate in the window (§5.6). Under normal conditions, with no competition, the aggregator's candidate is the only candidate and close is finality — the semantic difference appears only under attack. **Withdrawal and L2→L1 bridging effects execute only from window-final state**, never from provisional state.

- Batch size is bounded above by proving cost, and by how much body data can be published to L1 before the candidate lands; it is no longer bounded by what one transaction can carry. Batches need not align with epoch boundaries.
- **Canonical blob binding**: bodies use the signed manifest, SSZ encoding, KZG transcript binding, publisher attribution, TTL and bounded-ring storage specified normatively in §6.3. A candidate exposes at most $B_{\max}$ ordered versioned hashes, and the circuit proves that their canonical decoded bodies match the signed block headers and state transitions. `postData` is append-only only within its bounded retention version; expired data may be reposted under a fresh publisher record.
- **Two consequences, and one non-consequence**. The throughput ceiling is gone: the landing transaction carries a proof, a reference list and the end-state tuple, so the chain's advancement per settlement window is bounded by L1's blob bandwidth over the window rather than by what fits in one transaction — with 150 L1 slots per window that is roughly two orders of magnitude more room than a single transaction affords. **Data is also no longer paid for twice**: because references are permissionless, a later candidate in the same window reuses the blobs an earlier one already posted and pays only for its own increment, which is what makes a fallback lander's marginal cost small when it extends an aggregator's published prefix. The non-consequence is equally important: **this does not give the unlanded tail protocol-level availability**. Data reaches L1 only when someone intends to land it, so the deep-reorg residual of §5.4 and the preconfirmation DA assumption of §5.5 are unchanged. Publishing bodies continuously as blocks are produced is available as a strengthening that would close both, at the cost of paying for data that may be orphaned; it is a separate decision and is not assumed here.
- Referencing another publisher's registered blob is permitted and pays that publisher once if a successful fallback consumes it. Proofs remain beneficiary-bound. Data and proof attribution are separate public inputs.
- **Future slots must not be landed**: the `slot` of a batch's last block must be $\leq$ the `L2_slot` corresponding to the timestamp of the L1 block in which it lands (the permitted clock skew is initially 0) — checked at the L1 entry point and proven identically in the circuit. The lookahead is published at least $H_{\text{look}} = 768$ slots in advance; without this constraint a builder could pre-sign blocks for future slots, and a batch could push the landed chain head ahead of wall-clock time — destroying the timestamp/anchoring/forced-expiry semantics and artificially driving the `lag` of §6.3 negative. Correspondingly, the arithmetic definition of `lag` is truncated below at 0 (safe arithmetic).

6.2 Cadence

- Target landing cadence: the aggregator should keep the lag lag defined in §6.3 (the number of slots by which the tip of the best provisional candidate trails L1 wall-clock time — observable on L1, referring to no P2P state; lag is measured on the provisional candidate, so that the liveness determination is not dragged down by window delay) within Δ_{lag} . A proving latency of about 10–15 minutes means that in steady state provisional landing trails the live chain head by about 15–20 minutes; window-final finality adds one further W_{settle} — the decoupling of the preconfirmation experience (seconds-scale) from the finality cadence (minutes-scale) is unchanged, and it is the former that users perceive. The ladder of four levels of certainty, and the two lags:

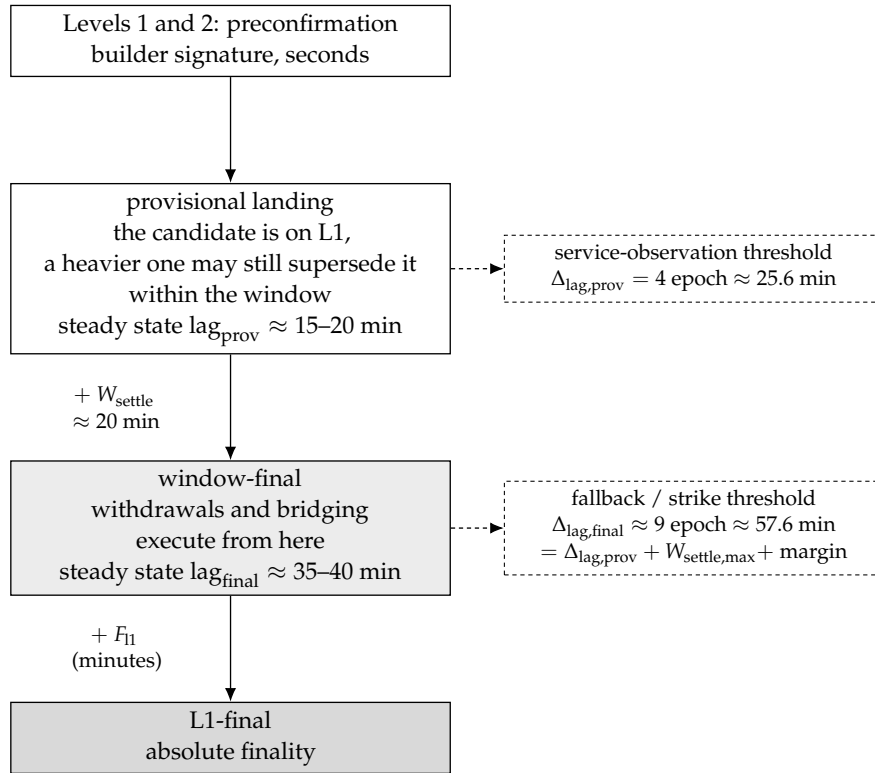


Figure 6: The four-level certainty ladder and the two lags: preconfirmation is a matter of seconds, and provisional landing is one settlement window away from window-final. lag_{prov} serves only to observe the service tempo, while it is $\text{lag}_{\text{final}}$ that enters fallback and slashing accounting.

- L2→L1 bridge (withdrawal) delay = landing cadence + W_{settle} (withdrawals execute only from window-final state, §6.1).
- **No fast close (fast path):** even when a window contains only a single candidate, the full W_{settle} is awaited before finality — a test for "closing early" would reintroduce gameable timing, and v1 does not do it; it is listed in §11 as an optimization (for example, "a candidate covering all known signature-chain heads may close early" may be introduced only after it has been proven not to be manipulable by block withholding).
- Two lags, each with its own role ($\text{lag}_{\text{final}}$ = the amount by which the window-final head trails — used for safety/revocable- exposure accounting and for fallback accounting (the

next item and §6.3). Two thresholds, in one-to-one correspondence (splitting only the adjudicated quantity without splitting the threshold would leave the fallback window permanently open): $\Delta_{\text{lag,prov}}$ (= the old Δ_{lag} , initially 4 epoch ≈ 25.6 min, setter invariant $\geq$ the upper bound of the normal provisional lag band) applies to lag_{prov} ; $\Delta_{\text{lag,final}} = \Delta_{\text{lag,prov}} + W_{\text{settle_max}}$ (wall-clock conversion) + margin (initially ≈ 9 epoch ≈ 57.6 min. 2 min, but $\Delta_{\text{lag,prov}} + W_{\text{settle_max}} = 128 + 150$ L1 slot = 55.6 min already exceeds it, so 8 epoch violates the very formula on this same line; once model property P9a was made non-vacuous (deployment values declared independently, inequalities cross-checked) this was caught immediately, and the value was changed to 9 epoch = 288 L1 slot, with a margin of 2 min) applies to $\text{lag}_{\text{final}}$. Rationale: under window finality the steady-state $\text{lag}_{\text{final}} \approx \text{lag}_{\text{prov}} + W_{\text{settle}} \approx 35\text{--}40$ min, which by itself already exceeds the old threshold of 25.6 min — if fallback/strikes were adjudicated on $\text{lag}_{\text{final}}$ while the old threshold were retained, the fallback window would be permanently open in a completely healthy system, the "cost + markup" reward commitment would remain in force in perpetuity, and a diligent aggregator would accrue strikes continuously until termination; the current rule the adjudicated quantity without raising the threshold is precisely this bug. The honest bound on revocable exposure: although a provisional candidate has already landed on L1, it can still be superseded before close, so the worst-case revocable set of tier-2 preconfirmation = the unlanded tail + the provisional increment within the window, and the true bound = $\Delta_{\text{lag,prov}} + W_{\text{settle}}$ (wall clock) + fallback response $\approx \Delta_{\text{lag,final}} + \text{fallback response}$ — it can no longer be written as " $\approx \Delta_{\text{lag}}$ " on its own; elsewhere in this document, the Δ_{lag} appearing in depth bounds of the form "depth $\approx \Delta_{\text{lag}} + \text{fallback response}$ " is, per to be read as the fallback-window-opening threshold $\Delta_{\text{lag,final}}$ (a new name for the same quantity; the bound is unchanged, only the naming is made honest). In order to pin this bound down, W_{settle} is given, besides its lower bound, a consensus upper bound $W_{\text{settle_max}}$ (§11; including the margin for wall-clock conversion when L1 slots are missed), which is folded into the user-facing semantics and the wording of §5.4.

6.3 Landing SLA and Versioned Permissionless Recovery

The fallback is one persistent episode in the same state machine as normal settlement. There are no rounds, commitments, target relaxation, winner-identity attribution, or slash-funded bounty.

```

sync(now):
  minAdmissibleSlot := max(minAdmissibleSlot, currentL2Slot(now)-H_slot_expire)
  if mode==NORMAL_OPEN and now >= closeAt: commitNormalBest(); mode := NORMAL_IDLE
  if mode != FALLBACK_OPEN and finalLag(now) > Delta_lag_final:
    outgoing := activeAggregator; burn(outgoing.penaltyBond); terminate(outgoing)
    promoteFirstFullyFundedStandbyOrLeaveSeatVacant()
    version := version+1; activationSlot := currentL2Slot(now)
    activationId := keccak(protocolVersion,chainId,this,block.number,version,
      blockhash(block.number-1)); mode := FALLBACK_OPEN
    if normalBest != NONE and qualifies(normalBest.tipSlot,now): commitFallback(normalBest)
    else: cancelNormalWindowAndItsHandles()

qualifies(tipSlot,now): tipSlot >= currentL2Slot(now)-Delta_close

submit(candidate,expectedVersion):
  sync(block.number); require(expectedVersion==version)
  if mode==FALLBACK_OPEN:
    require(qualifies(candidate.tipSlot,block.number)); require(candidate.base==canonical)
    require(candidate.first.isTierII and candidate.first.slot >= activationSlot)

```

```

    require(candidate.first.(fallbackVersion,fallbackActivationId)==(version,activationId))
    verifyProofAndRegisteredData(candidate); commitFallback(candidate)
else: acceptNormalCandidate(candidate)

```

- **Unique ordering at every entry point:** every state-mutating entry point calls sync first. A mature normal window closes before lag is recomputed. If its committed winner cures the lag, fallback never activates. If the deadline is still breached, activation has priority over accepting a new normal candidate. An already-verified normal best that satisfies the acceptance-time target is committed as the first fallback result; otherwise activation cancels it and increments the version, so stale close or submit handles revert. This removes the same-height ambiguity.
- **Strict service liability:** crossing $\Delta_{\text{lag},\text{final}}$ terminates the incumbent and burns its penalty bond, independently of who later proves recovery. The protocol cannot permissionlessly distinguish an honest global proving outage from collusion or poor operations; the aggregator prices that risk into its bid. There is no outage oracle and no zero-false-positive claim. This objective rule also prevents an incumbent from repeatedly front-running external fallback transactions while keeping the seat: it is already terminated before a recovery submission is processed.
- **First qualifying proof:** while FALLBACK_OPEN, only a qualifying proof is a candidate transition. A sub-target proof is rejected; body publication remains a separate postData action and changes no canonical state. The first qualifying proof in L1 transaction order atomically commits the complete canonical tuple, records `canonicalizationBlock=block.number`, pays once, increments the version, and returns to NORMAL_IDLE. A later heavier tail cannot supersede it. Fallback therefore explicitly revokes every omitted tier-2 preconfirmation and returns those transactions to the mempool.
- **Moving-target feasibility:** $H_{\text{recovery},\text{max}} + 12 \times (P_{\text{prove},\text{max}} + T_{\text{include},\text{max}} + \text{clockMargin}) \leq \Delta_{\text{close}} < 12 \times \Delta_{\text{lag},\text{final}}$. Under the in-model requirement of one honest scheduled ticket in every fixed window, $H_{\text{recovery},\text{max}} = 2 \times W_{\text{size}} = 768$ L2 slots: activation may occur just after the current window's last honest ticket and the next window's honest ticket may be last. Initial $\Delta_{\text{close}} = 2400$ slots (40 minutes) therefore covers that scheduling wait plus a demonstrated proof/inclusion path of at most 95 L1 slots. Governance cannot set a value outside the relation; recursive continuation is not assumed.
- **Episode-bound recovery:** the first fallback block is a tier-(ii) block signed after activation, at a scheduled non-VACANT slot at least `activationSlot`. Its header binds the version and `activationId`, which commits to the protocol domain, contract, activation height, version and preceding L1 block hash. It extends the current canonical head and may jump to the wall clock. This prevents pre-signing, historical zero-bond backfill and cross-episode replay.
- **Transparent-mempool limit:** the terminated incumbent or promoted standby may still pre-empt one outside prover by submitting its own valid proof first. Doing so restores finality, and the outgoing incumbent cannot repeat the attack without another penalty bond and seat. The design does not promise that every losing prover recovers sunk cost; it promises that no party can retain the failed seat by winning the race. Operators should use early public proof jobs or private order flow to reduce duplicate work, but liveness does not assume private order flow.

Recovery funding and blob binding. The outgoing seat pre-funds a separate R_{fallback} escrow before activation. It is refundable even after termination except for successful recovery payments, and is never slashable; the penalty bond is burned in full. The locked escrow for an episode must cover $R_{\text{proof}} + B_{\text{max}} \times R_{\text{blob,max}} + R_{\text{submit}}$. If it does not, the seat cannot activate in the first place. A newly promoted standby must independently satisfy the same funding invariant before it becomes active.

A blob uses the canonical SSZ body encoding and a manifest (`protocolVersion`, `chainId`, `bodyRoot`, `publisher`, `validUntil`). The publisher signs the manifest; the blob carries the manifest and signature, and `postData` verifies that the caller is the signed publisher before recording (`versionedHash`, `bodyRoot`, `publisher`, `blobBaseFee`, `validUntil`, `paid=false`). The proof exposes the KZG evaluation transcript and proves that the decoded body hashes to `bodyRoot`; the L1 verifier checks the point-evaluation precompile result against the registered versioned hash. This is the sole consensus binding, not a calibration choice.

At fallback commit, the contract pays the fixed R_{proof} and R_{submit} to the proof beneficiary, and for each newly consumed registered blob pays $\text{BLOB_GAS_PER_BLOB} \times \text{recordedBlobBaseFee}$ to that record's publisher, capped by $R_{\text{blob,max}}$; then it marks the record paid. A candidate may reference at most B_{max} blobs. Records expire after `DATA\_TTL`; a publisher may repost the bytes under a fresh record after expiry. Storage is a version-scoped bounded ring retained through fallback/normal finality plus the L1 reorg horizon, then reclaimable. Base-fee share φ_{land} is credited in the same atomic fallback commit. No payment is made from a slash, no undefined coverage term exists, and a winner cannot collect another publisher's data cost.

6.4 Stall and Gap Recovery

A normal P2P gap of at most G_{max} is crossed by the next scheduled builder. For a longer stall, a scheduled builder may sign a tier-(ii) block on a canonical head only after that head's `canonicalizationBlock` has reached F_{11} depth. Finality depth is measured from the normal close or fallback commit that made the tuple canonical, never from an earlier provisional candidate transaction.

When fallback activates, the recovery block described in §6.3 deliberately re-roots at the canonical head and reconnects near the wall clock. The old unfinalized tail is not guaranteed to land first: a first-valid fallback commit may discard all of it. This is the explicit price of bounded recovery without an unenforceable heaviest-chain oracle. After the commit, ordinary tier-(i) production continues when scheduled builders return at least once per G_{max} ; sparse participation below that floor can keep producing isolated recovery attempts but cannot sustain the chain.

Anchor freshness still bounds proving and inclusion. A recovery block that misses $D_{\text{anchor,max}}$ must be re-signed at a later scheduled slot with a fresh anchor and re-proved. The setter relation in §11 covers the full SLA, proof and inclusion path. The L1-to-L2 message cursor reconnects in the recovery block and drains the deterministic maximum prefix under the per-block count and gas caps; the anchor itself may jump directly to a recent authenticated L1 header.

6.5 Aggregator Seat and Economics

- The seat is produced by a perpetual auction (inheriting the v15 §4 skeleton: bonded bidding, the standby queue, the q delayed transition, bonds, and re-auction upon T_{max} expiry — all carried over, with the semantics of q and T_{max} unchanged).

- **The fee source must not form a circular dependency with liveness:** if the service fee comes from the protocol fee stream, then when the chain is idle the fee stream dries up and creates the loop "underpayment $\rightarrow$ rationally going offline $\rightarrow$ dependence on the fall-back"; the fallback reward is therefore paid solely from the aggregator-funded reward escrow rather than from the protocol fee stream (§6.3; the escrow is separate from the penalty bond, and refundable), and the design of the service-fee source (§11 item 4) must guarantee that the aggregator still breaks even when the fee stream is insufficient (for example, a minimum service fee backstopped by the treasury). The combination of winning the seat cheaply and then slacking off is already closed off by the strike mechanism of §6.3 (slacking off = strikes = replacement), so all that needs to be handled here is the "honest but underpaid" case.
- **The direction of payment is reversed, stated explicitly:** in v15 the seat paid the treasury because it held an exclusive sequencing right; in this design the seat is a service post (batching + proving + L1 gas), and what is auctioned is the lowest service fee rate (a reverse auction, with the service fee paid out of protocol fee revenue), with the bond accounted for separately. The bidding dimensions are the pair (service fee rate, bond): the lower rate wins, and at equal rates the higher bond wins. Refinement is a calibration item of §11.
- MEV and priority fees go to the builder of each slot (coinbase). The aggregator does not touch sequencing and therefore structurally has no MEV position — this is intentional.
- **Base-fee share:** in normal mode, a fixed ratio φ_{land} of the winning candidate's aggregate base fees is credited to its proof beneficiary at close. A superseded candidate receives none, so a longer proven chain has both the larger ordering key and the larger fee base. In fallback, the first qualifying proof commits immediately: φ_{land} still grows with included content, but speed can beat completeness and omitted tier-2 promises expire. All beneficiary fields are proof-bound, and all accounting is atomic with the canonical commit; provisional submissions are never paid and require no clawback.

7 Anchoring and the bridge (L1 $\rightarrow$ L2)

- The current per-block anchoring pattern is carried over: the first transaction of every block is the anchor transaction (anchor tx), which references an L1 block at depth $\geq D_{\text{anchor}}$ (32 L1 slot), is non-decreasing relative to the previous anchor, and whose freshness does not exceed the cap (the numerical values carry over the shape of the v15 §6.6 constraints).
- **The anchor may not be later than this block's slot time — the L1 $\rightarrow$ L2 causal-ordering invariant :** in addition to being non-decreasing and fresh, it must also hold that the anchor's L1 timestamp $\leq$ the L2 timestamp corresponding to this block's slot (equivalently, $L2_{\text{slot}}(\text{anchor.L1_timestamp}) \leq \text{slot}$), and this applies to all blocks alike. Why it is necessary: §6.1 rule 3 only blocks "future slots", and §7 only blocks "an anchor that is too old", and neither blocks "an old slot paired with a new anchor" — a builder could hold back an old slot, then pick after the fact a fresh anchor that has just arrived and that already contains some new L1 $\rightarrow$ L2 message, and sign a tier (i) continuation that consumes that message into an L2 block whose slot timestamp is earlier than the L1 block the message originated in; a Byzantine aggregator lands it ahead of honest recovery $\rightarrow$ L1 $\rightarrow$ L2 causal ordering is broken, and the bridge/message deadline semantics based on `block.timestamp` are distorted. With this invariant added, a block can never reference L1 state from after its own slot time. It does not affect recovery: a recovery block refer-

ences an L1 block at depth $\geq D_{\text{anchor}}$ (≈ 6.4 minutes ago), whose timestamp is always $\leq$ the current wall-clock slot time, so an honest block satisfies it naturally; this invariant rejects only the malformed combination "old slot + new anchor". The L1 entry point and the circuit both prove it (anchor.L1_timestamp is a pure function of the referenced L1 block header).

- The time geometry of the anchor's age along the worst-case path (the origin of the $D_{\text{anchor_max}}$ setter invariant, Appendix C P9a):

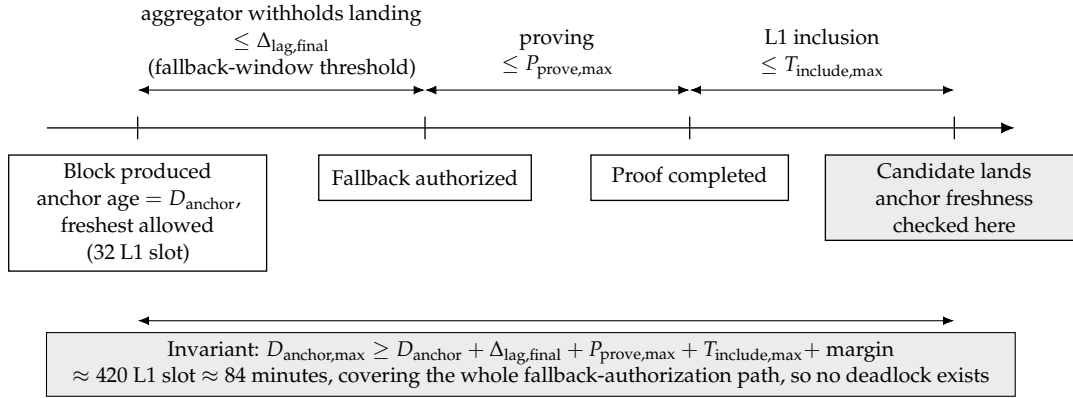


Figure 7: The time geometry of anchor age along the worst-case path: the freshness window $D_{\text{anchor,max}}$ must cover the whole route from block production, through the aggregator's withholding until the threshold authorizes fallback, and on to proving and L1 inclusion. This relation between parameters is exactly the solution to the stale-anchor tail deadlock.

- **The solution to the stale-anchor tail deadlock — $D_{\text{anchor_max}}$ covers the fallback authorization period (the commitment layer has been deleted):** when a malicious aggregator withholds landing for a long time, the anchor of an early block of an honest tail will, "at landing time", exceed $D_{\text{anchor_max}} \rightarrow$ the tail can then neither be landed (its first block is past the deadline) nor be skipped (parent_hash runs through it). The solution is parameter geometry, not a new mechanism: the setter invariant $D_{\text{anchor_max}} \geq D_{\text{anchor}} + \Delta_{\text{lag,final}} + P_{\text{prove,max}} + T_{\text{include,max}} + \text{margin}$ (the lag term is $\Delta_{\text{lag,final}}$ — the fallback opens its window on lag_final , so the authorization point is later than under the old Δ_{lag} , and the freshness window must cover it, or else the C1 fix would in turn break this geometric solution) — that is, the freshness window must cover the entire course "from block production, through lag_final crossing the threshold and opening the fallback, to the fallback proving and landing it". Consequently, any tail that the fallback is authorized to land (produced while $\text{lag_final} \leq \Delta_{\text{lag,final}}$ and landed by the fallback after the threshold is crossed) necessarily still has a fresh anchor, and no deadlock exists. A rough calculation with the initial parameters: $32 + 288 + 75 + 10 + \text{margin} \approx 420$ L1 slot (≈ 84 minutes), which is still a deployable value. For situations beyond that window (the fallback too is persistently absent for $> D_{\text{anchor_max}}$), the stale prefix can no longer be landed and the chain is rebuilt from the window-final head — the same conditional residual risk as §6.3 fallback feasibility, stated honestly. The causal-ordering invariant (the previous item) is unaffected.
- Because there is a block every second (under normal conditions), the intake cadence of L1→L2 messages is on the order of seconds — better than the fallback path of a forced-

inclusion-specific cadence. During a gap, messages are consumed by the next honest block; inbound messages have no alternative entry route under a full cartel and can only wait for scheduled builders to resume producing blocks (§8).

- **The L1→L2 message-processing cap C_{anchor} — bound to the [processing cursor], not to the anchor reference** : distinguish two independent quantities:
 - **The anchor reference** (which L1 block the block references as its freshness baseline): still non-decreasing + required to be fresh ($\text{landing L1 block height} - \text{anchor} \leq D_{\text{anchor_max}}$). A recovery block references a current, fresh anchor — even if the anchor of its parent (a landed head from days ago) is very old, this one large monotone jump is permitted (it merely references a recent L1 block and satisfies freshness), and the anchor itself carries no per-block advance cap.
 - **The L1→L2 message-processing cursor m_{consumed}** (hence m_{consumed} = the global sequence number in the inbound message queue, or the equivalent (L1 height, tx index, log index) triple): the unique canonical algorithm = [the uniform maximum prefix]: L1→L2 messages form a public FIFO queue (the contract maintains a queue root + tail cursor, kept separate from m_{consumed}); every block must consume the longest prefix of that FIFO that satisfies both "entry count $\leq C_{\text{anchor}}$ " and "cumulative declared gas $\leq$ the L1→L2 message gas share" (restricted to those that have arrived within the height referenced by this block's anchor and are unconsumed; taking that longest prefix is legal, and only taking less than it is illegal — as in §5.1 rule 4, with no second "exactly some entry count" criterion). A recovery block references a fresh anchor (spanning days of L1), each block digests this prefix, and the remainder is continued by subsequent blocks (m_{consumed} advances monotonically). The backlog catch-up bound is correspondingly two-constraint: $\text{backlog drain time} = \max(\text{ceil}(\text{entry count} / C_{\text{anchor}}), \text{ceil}(\text{cumulative gas} / \text{the } G_{\text{l1msg}} \text{ share}))$ blocks (the bounds of §6.4/§8 are to be read this way). Without this lower bound, m_{consumed} is monotone yet may stagnate: a builder that keeps processing 0 messages while still using a fresh anchor could censor L1→L2 messages indefinitely, invalidating the backlog / C_{anchor} catch-up bound of §6.4/§8. With the lower bound added, the catch-up bound is circuit-enforced and redeemable. Consumption is not execution — an inbound message that is illegal against the preceding state is consumed-and-discarded under the same consume-and-discard rule as §7. **The real anti-censorship bound is $D_{\text{anchor_max}}$, not C_{anchor} alone**: the maximum prefix only forces "consumption of those that have arrived within the height referenced by this block's anchor", so a builder that pins its anchor at the oldest height freshness permits ($\text{landed head} - D_{\text{anchor_max}}$) can legally consume no newer message at all. But this is not indefinite censorship: the anchor must satisfy the freshness lower bound $\text{anchor} \geq \text{landing L1 block height} - D_{\text{anchor_max}}$ and be non-decreasing, so as the landed head advances, that lower bound rises, and a message that arrived at height h is forced into "within the anchor-referenced height" once $\text{landed head} - D_{\text{anchor_max}} > h \rightarrow$ it must then be consumed. Hence the worst-case inclusion delay of an L1→L2 message $\approx D_{\text{anchor_max}}$ (freshness forces the anchor frontier forward) + $\text{backlog} / C_{\text{anchor}}$ (the processing rate once it has entered the to-be-consumed set); the two parameters jointly bound it, not C_{anchor} alone. Why it is split this way: the §4.2 tier (ii) recovery block has to reconnect to the wall clock in one jump from a landed head days old. If C_{anchor} were imposed on the anchor reference, then the anchor of a recovery block could advance only C_{anchor} from its

stale parent anchor, would still be stale, and would fail the freshness check — failing precisely on the long stall it is meant to support. Once it is rebound to the message-processing cursor: the anchor reference is fresh (satisfying §7) and L2 ordering recovers in one jump; while a huge volume of L1→L2 messages is processed block by block at C_{anchor} per block, so a full sync takes $\text{message backlog} / C_{\text{anchor}}$ blocks (after recovery, 1 block/second, which is fast). This is the same technique and the same shape as a fresh reference plus a rate-limited cursor. C_{anchor} is a §11 calibration item.

- **A shared block-level gas budget + a per-message gas cap — sealing off the combined-gas chain stall:** the original risk was that the "forced prefix" and "L1→L2 message consumption", two mandatory obligations each with its own quantity, could together burst the block gas cap and leave no legal block even with everyone honest. With the forced prefix gone that combination disappears, but **one side alone can still reproduce a permanent stall, in a milder form:** if the head of m_{consumed} is an inbound message whose declared gas exceeds "the block cap minus the fixed anchor overhead", the maximum prefix that fits is empty at every subsequent block. Blocks stay legal — rule 4 asks for the longest prefix that fits, not for a fixed count — but m_{consumed} can never advance past that message, so the inbound message queue is blocked for good while the chain itself continues. The damage is confined to the bridge rather than to liveness, which is why the per-message admission cap below is the clause that carries the weight (model property P7b constructs exactly this state and confirms the cursor stalls). The first two clauses below are therefore retained, and clauses 3/4 restated for two parties (fully deterministic, verifiable by both the circuit and L1):
1. **A per-item admission gas cap:** inbound L1→L2 messages get a per-item gas cap at enqueue time, and an over-cap item is refused admission — this is the load-bearing clause against one-sided deadlock, and it is not relaxed just because the forced queue is gone;
 2. **A shared block-level budget invariant:** the fixed anchor overhead G_{anchor} + the L1→L2 message gas share $\leq \text{block_gas_limit}$, as a setter invariant on the consensus constants; the message side's "guaranteed capacity after deducting the anchor overhead" always fits into the block;
 3. **Deterministic priority and overflow:** the enforced execution order within a block = anchor → L1→L2 message prefix; following m_{consumed} , the block "consumes up to its guaranteed capacity, and the remainder deterministically overflows into subsequent blocks" — the cursor is advanced only by legal blocks, so a backlog buys delay and cannot buy a chain stall;
 4. **A watermark for lowering parameters:** when C_{anchor} or the message gas share is lowered, the new share must be $\geq$ the maximum gas among the unconsumed entries of the message queue, guaranteeing that in any reachable state there is no item that is "already enqueued yet unable to fit within the guaranteed capacity". The two separately rate-limited obligations are thereby coordinated by one shared budget, and can no longer be combined into a "no legal block" deadlock. The numerical gas shares are folded into §11.

8 What happens when each role goes offline (liveness accounting, in a single table)

Failure	Protocol response	User-visible bound
A single builder	Its ticket becomes a gap. The next non-VACANT builder resumes.	At most two seconds from that builder alone, including a window boundary.
Up to 63 consecutive absences	The next builder resumes under tier (i).	The number of absent slots, in seconds.
At least $G_{\max}$ consecutive absences	A fresh, episode-bound tier-(ii) block can reconnect from the final canonical head; it may discard the unfinalized tail.	F_{L1} finality wait plus demonstrated full-proof and L1-inclusion bounds. Sustained progress still needs one scheduled builder per $G_{\max}$.
Aggregator offline or Byzantine	At the objective final-lag breach, sync burns and terminates the incumbent, promotes a fully funded standby if one exists, and opens one persistent fallback episode.	$\Delta_{\text{lag, final}}$ plus full proof and bounded inclusion, conditional on a landable builder chain and a funded prover.
No standby	The seat stays vacant; fallback stays open and the auction may fill the seat.	Sequencing continues. Finality recovery still requires a funded external prover.
All builders absent or colluding	No scheduled key produces a block and there is no forced path in this revision. New entrants appear only after the snapshot delay and window alignment.	Unbounded ; queued user transactions have no protocol entry path. This is outside the stated trust model and blocks a strong-permissionlessness claim.
Global proving failure	One incumbent is burned and terminated; the persistent episode remains open without successively burning standbys.	Outage duration plus proof and inclusion. This is an operator-insured risk; no sound outage oracle is assumed.

By comparison with v15: there, "one seat goes offline" = a 20–26 minute service interruption (when a standby exists); here the same class of event costs 1 second (a builder) or 0 seconds (an aggregator, which affects only the cadence of finality). This is the reason this design exists.

9 Master table of slashing and bonds

Fault	How it is adjudicated	Consequence
Builder equivocation (two headers for one slot)	Direct canonical ECDSA verification on L1	Burn the independently reserved L_{lease} for that schedule window in full; fixed reporter reimbursement comes from protocol fees; deactivate future tickets
Builder skips the previous block / is absent	Not a fault (producing a block is an opportunity, not an obligation)	No slashing; a skip is profitable for the skipper (fee/MEV transfer). Orphaning depth follows the three tiers of §5.4: an adjacent skip affects a single slot; under tier (i) a single-block deep skip orphans at most $G_{\max} - 1$ unlanded blocks, and a minority coalition relaying a sparse branch can reach the entire unlanded tail $\approx \Delta_{\text{lag}}$; under tier (ii) a single malicious builder can, with one block built on a final landed head, orphan at most the entire unlanded tail $\approx \Delta_{\text{lag}}$. The worst-case depth is the same in all three tiers, namely $\leq$ the actual length of the unlanded tail ($\approx \Delta_{\text{lag}}$ when the fallback lands on schedule, conditional); in every tier the attacker must beat any heavier honest candidate inside the §5.6 window before it can become final, the signature is on record for public attribution, and the frequency is constrained by the honest-majority assumption; an absentee forfeits the fee income of its own slot

Fault	How it is adjudicated	Consequence
Builder signs a header but withholds the body (withhold-body, §5.5)	Not adjudicable ("it has the body and is not releasing it" cannot be proven)	No slashing; the block is certain to be skipped and the withholder gains nothing at all; the receipt is itself public evidence, and the response is reputational
Aggregator SLA breach ($\text{finalLag} > \Delta_{\text{lag, final}}$)	sync evaluates the L1 clock after closing any mature normal window	Immediate termination and full penalty-bond burn; promote a fully funded standby or leave the seat vacant; one persistent fallback episode opens
Aggregator takes a prefix or lands a worse chain	Not separately slashable	In normal mode a heavier proof may supersede it. If the objective SLA is breached, the seat is terminated and the first qualifying fallback proof may finalize a thin chain; omitted tier-2 promises expire
Anyone submits an invalid batch	The proof fails verification and the transaction reverts	Wasted gas, no protocol consequence
Builder signs a block that cannot execute (a body that does not apply, or a state root that does not match)	Not adjudicable on L1 without a proof of invalidity, which this design does not define; the block simply fails to be provable	No slashing. The block can never land, so it cannot corrupt state, and the signer forfeits its own slot revenue and gains nothing — the same shape as body withholding. The residual cost is borne by others: honest successors that have not yet executed the body may build on it within the detection latency, and those blocks are stranded when the next builder re-roots past it under §4.2 tier (i). Unlike equivocation this costs the actor nothing and is repeatable in every slot it is scheduled for, bounded only by its lookahead share w_{max} . A preconfirmation issued for such a block is void, and its holder cannot tell from the signature alone

Every slashable fault is adjudicated by direct L1 signature verification or an objective L1-clock predicate; none requires an L2 evidence chain or arbitration. A builder signature proves authorship and parent choice, not execution. Execution is established only by the validity proof, so skipping, body withholding and an unexecutable signed block remain non-slashable. The protocol contains their state impact but cannot truthfully promise to compensate the affected preconfirmation holder.

10 Detailed comparison with v15 (for readers who have read v15)

- **What became of v15's nine invariants:** I1 (derivation is a pure function) is inherited and strengthened; the spirit of I2 survives as objective L1-clock adjudication, but there is no epoch certificate; I3 becomes permissionless proof submission, subject to the builder/prover assumptions stated in §1; I6 has no counterpart because this revision has no forced inclusion; I7 becomes "landing carries no authority"; I8 survives as proof-beneficiary and signed blob-publisher attribution; I9 survives as ETH-denominated, independently reserved L_{lease} tranches and the aggregator penalty bond.
- **v15's review legacy is not voided:** the auction skeleton (§4), the at-fault-pays model (§6.7), the anchor freshness constraint (§6.6), the burn-dominant split of slashed funds (§8) and the discipline of invariant checks in parameter setters are all carried over directly. The parts of v15 that revolved around "one irreversible adjudication per epoch" (EBC, AC, default derivation, anarchy mode, cancellation cascade) have no counterpart in this design, so the corresponding historical review findings do not apply.
- **Response to the four criticisms in the assessment document:** the single point of fail-

ure is merely relocated → resolved (landing carries no authority, plus lag-based fallback and strikes, which are bounded for both the offline and the Byzantine aggregator — §6.3); builder faults are all of the watchtower-dependent class → the slashable class is resolved (equivocation is verified directly on L1), but two classes of non-slashable misbehavior (skipping, body withholding) remain as deterrence-grade residual risk and are stated explicitly (§5.4/§5.5) — this is "downgraded and explicitly priced", not "eliminated"; fair exchange has become load-bearing → partially resolved (intermediate deletion is eradicated by the signature chain; the residual risk is as above); the epoch mechanism needs a rewrite → conceded, and this document is that rewrite, whose result is shorter than v15.

11 Implementation gates and fixed decisions

This revision fixes the consensus choices that previously appeared as alternatives. It is still not deployable until the measurable gates below pass; a gate is not permission to substitute a different consensus rule.

1. **Proof benchmark:** implement the complete v1 circuit with ECDSA-secp256k1 header verification, the exact quota schedule, canonical blob/KZG binding, message-cursor execution and a full independently generated fallback proof. Demonstrate $P_{\text{prove,max}}$ on the supported hardware matrix. The protocol does not rely on recursive continuation or another prover's private state; either optimization may be added only after it benchmarks without changing public inputs.
2. **Economic calibration:** measure R_{proof} , R_{submit} , $R_{\text{blob,max}}$, $V_{\text{slot,max}}$, L_{lease} , R_{fallback} , normal W_{settle} , $H_{\text{recovery,max}}$, Δ_{close} and $\Delta_{\text{lag,final}}$. Setters enforce all lower and upper relations atomically. A fee market above the published caps is outside the economic liveness guarantee, but cannot corrupt safety.
3. **Contract and circuit benchmark:** bound the active registry at 64, candidate blobs at B_{max} , schedule windows per candidate, public inputs, storage-ring retention and gas for sync, normal replacement, fallback commit, KZG checks and tranche release. The exact algorithm is fixed; only its implementation strategy is benchmarked.
4. **P2P profile:** specify canonical receipt exchange, block/body propagation, retry, peer scoring and public telemetry. The protocol still assumes at least one non-colluding scheduled builder exposes a landable chain and one funded prover accepts the fallback job.
5. **L1 reorg semantics:** write the atomic replay/rollback rule for the canonical tuple, mode/version, normal best, activation, seat change, burns, payouts, data-paid bits, liability watermarks and tranche release. Tier-(ii) finality is measured from `canonicalizationBlock`. This remains blocking in Round 1.
6. **Genesis and migration:** initialize one L1-final canonical tuple, registry/schedule version, mode/version, active and standby seats, escrow and watermarks; suppress SLA enforcement until the activation checkpoint; disable the old and new landing entry points atomically. This remains blocking in Round 1.
7. **Strong permissionlessness decision:** the current capital-ranked 64-key builder set, honest-builder assumption and absence of an L1 forced escape path do not provide strong permissionlessness or censorship resistance against a stake cartel. Either accept that product classification explicitly or add an L1 escape path before deployment. This is a product/security

boundary, not something the address cap solves.

8. **Human review and model parity:** every consensus edit must change `settlement-window-model.py` or `lookahead-model.py` in the same commit. Passing models are regression evidence, not a proof. Independent review must challenge the newest commit, including transparent-mempool races and same-height EVM traces.

Master parameter table

Parameter	Initial value and unit	Key invariant / source
slot	1 second	§3.1
H_slot_expire	4096 L2 slots	Advances <code>minAdmissibleSlot</code> independently of proof progress; exceeds authorization, proof, inclusion, reorg and evidence margins.
H_recovery,max; Δ_{close}	768; 2400 L2 slots	Builder scheduling wait plus full proof and inclusion is no more than Δ_{close} , which remains below $\Delta_{lag,final}$.
R_proof, R_submit	benchmarked ETH caps	Paid to the successful proof beneficiary from the outgoing seat's escrow.
R_blob,max, DATA_TTL	benchmarked ETH / L1 slots	Per-blob reimbursement cap and bounded signed-publisher record lifetime.
B_max	6 blobs	Consensus cap on verifier work and fallback data reimbursement.
R_fallback	fully funded ETH escrow	At least $R_{proof} + R_{submit} + B_{max} \times R_{blob,max}$; disjoint from the penalty bond.
epoch; W_size	384 L2 slots each	One fixed aligned lookahead window per epoch; §3.1/§3.2.
H_look	768 L2 slots	Published floor; the snapshot relation actually guarantees 1152 slots.
D_snap	5 L1 epochs	At least lookahead horizon + L1 finality + margin.
Q_max; N_max	76 tickets; 64 addresses	Per-address cap and registry cap. Under-capacity tickets become VACANT; six addresses are required to fill a window.
G_max	64 L2 slots	Tier-(i) parent-gap cap; a depth knob, not a liveness guarantee.
G_max_landed	unbounded	Tier-(ii) permits any gap only from an L1-final canonical parent.
F_l1	TBD L1 slots	Finality depth measured from <code>canonicalizationBlock</code> .
C_anchor	TBD messages/block	Maximum-prefix message consumption cap bound to <code>m_consumed</code> ; §7.
Δ_{prop}	TBD L2 slots	Normal landing-propagation settling depth; smaller than provisional-lag threshold.
δ_{slash} ; R_evidence	64 L2 slots; fixed ETH	Future deactivation delay and fee-funded reporter reimbursement.
$\Delta_{lag,prov}$	4 epochs	Service observation only; above the healthy provisional-lag band.
$\Delta_{lag,final}$	about 9 epochs	Final fallback/termination threshold; exceeds provisional lag + maximum settlement window + margin.
W_settle,max	TBD L1 blocks	Consensus upper bound on revocable exposure and wall-clock conversion.
L_lease	calibrated ETH/window	Net burn covers $Q_{max} \times V_{slot,max} \times safetyFactor$; independently reserved per schedule window.

Parameter	Initial value and unit	Key invariant / source
B_aggregator	calibrated ETH	Fully burned on the first objective SLA breach in a persistent episode.
D_anchor; D_anchor,max	32 / TBD L1 slots	Freshness cap covers final lag + proof + inclusion + margin.
W_settle	TBD L1 blocks	At least full proof + inclusion + margin; about 100 blocks is the pre-benchmark estimate.
φ_{land}	TBD ratio	Winning-chain base-fee share credited atomically at close or fallback commit.
Anchor/message gas shares	TBD gas	Their sum fits the block gas limit; each admitted message has a gas cap and a deterministic watermark.

(The values are initial suggestions; those marked "TBD" are calibration items under §11. Unit conversion: 1 L1 slot = 12 s = 12 L2 slot.)

Appendix A: Design alternatives considered and rejected

Each item states an option that was weighed against the choice this document makes, and why the choice went the way it did.

1. **The lookahead randomness does not mix in a proof hash.** A natural alternative is "L1 consensus-layer data + the hash of the most recent proof". This document takes only the former. The reason (§3.2): a proof hash is entropy that the party producing the proof can re-grind at zero cost, so mixing it in hands partial control of the lookahead to the aggregator, which runs counter to the trust-minimization goal. The 1-bit-level bias residual of pure L1 beacon randomness is acceptable for the purposes of the lookahead.
2. **Parent linkage uses the block-header hash, not the parent's signature.** Both are workable; this document makes the hash the consensus rule (it is structurally necessary and transitively commits to the entire history) and demotes carrying the parent's signature inside the child block to an optional evidence-packaging convention (§4.2).
3. **The residual risk from skipping is classified as deterrence-grade and explicitly accepted.** The signature chain eliminates intermediate deletion by the aggregator, but a skip by the adjacent builder is not falsifiable inside the protocol (§5.4). This document chooses to say so plainly and to handle it with publicity + reputation + a cap of one slot on the loss, rather than introducing an undecidable "but I did publish it" arbitration. If elimination at the structural level is wanted, the known price is publishing a commitment to L1 every slot (fees revert to what they were before aggregation saved them) — not recommended. (This sentence is precisely the cost identity cited in §1: the fee for buying protocol-level commitments or DA for the pre-landing stage $\approx$ the fee this design saves by landing in aggregated batches — buy the former and the latter's savings cease to exist.)
4. **The payment point for the base-fee share: the close, not provisional landing.** The obvious alternative is to distribute the base fee directly to the aggregator at landing time. This document chooses to account for it in the atomic commit at settlement-window close (§6.5). The reason: a provisional landing can be superseded inside the window by a heavier candidate (§5.6), so paying at landing would require designing a clawback mechanism for the superseded party — that is a new state machine and a new attack surface. Paying at the close, by contrast, is part of the atomic commit at the close, requires zero additional machinery, and happens to implement exactly the "winner takes all" incentive (a superseded lander receives nothing, which maximizes the opportunity cost of landing a short chain). The only price is that the lander's cash flow is deferred by one W_{settle} (≈ 20 minutes). Paying at landing would require accepting the complexity of a clawback state machine back into the design.

Appendix B: Glossary

Term	Definition
slot	The 1-second time unit of L2
lookahead	The deterministic mapping from slot to builder address
builder	The preconfir that produces and signs a block according to the lookahead
aggregator	The service seat that packs, proves and lands
signature chain	The unlanded chain linked by parent_hash and signed block by block
landing	A batch plus its proof going onto L1 as a candidate; at window close the heaviest becomes final
batch	A stretch of contiguous signature chain + data + proof
gap	A slot with no legal block
skip	A builder's signed choice not to build on the immediately preceding block that does exist
fallback landing	Anyone landing, and claiming compensation, once the aggregator has timed out
equivocation	Signing two different block headers for the same slot; slashed by direct verification on L1
two-tier parent rule	The criterion is the gap: (i) a gap $\leq G_{\max}$ (independent of the parent's landing status); (ii) no cap on the gap, but the parent must be an L1-final landed head
recovery block	A block that, during a stall, is built on an L1-final landed head under tier (ii) and reconnects to wall-clock time (§6.4)
benign stranding	When the parent tip of a recovery block is advanced, that block is voided and simply has to be rebuilt; no state is damaged (§6.4)
consume-and-discard	An inbound message that is illegal against the preceding state still advances the cursor but is not executed
best-chain total order	Lexicographic order on the candidate's own triple (count, tip_slot, tip_hash); the lane component retired together with forced-only blocks (§5.2)
best-chain landing strategy	In normal mode a rational lander submits the best visible chain because a heavier proof supersedes a worse candidate before close; fallback instead commits the first qualifying proof
settlement window	Opens once the first candidate extending the window-final head has landed, and closes deterministically W_{settle} L1 blocks later
candidate batch	A landed batch that extends the window-final head and carries a validity proof; inside the window it can be superseded by one that is strictly heavier under the §5.2 total order (§5.6)
window-final	The heaviest proven candidate at settlement-window close; withdrawals and bridging execute only from this state (§5.6/§6.1)

Appendix C: Executable consensus models

The dependency-free `settlement-window-model.py` is the single executable mode machine for normal settlement and fallback; `lookahead-model.py` is the exact production scheduling path. A test of a detached helper is not evidence about consensus. Every consensus edit changes the relevant model in the same commit. Passing assertions are regression evidence, not a proof or a substitute for independent review.

Property	Verified content
P1-P2	Normal candidate order is total/transitive; provisional replacement leaves canonical state unchanged; close commits the heaviest candidate exactly once.
P3	Every entry closes a mature normal window before recomputing lag, preventing an opposite same-height result.
P4	SLA activation burns and terminates the incumbent, promotes only a fully funded standby, commits an already-verified qualifying best, cancels a nonqualifying best and invalidates stale handles.
P5	Fallback rejects sub-target, old-version and non-episode-bound candidates; the first qualifying proof commits; the moving-target lower and upper parameter bounds hold.
P6	An outgoing incumbent may front-run once and restore finality but cannot retain the seat; one persistent proving outage terminates at most one incumbent in that episode.
P7	Proof/submit payment goes to the beneficiary, consumed-blob payment goes once to the signed publisher, active seats are pre-funded, and no slash-funded bounty exists.
P8	Tier-(ii) L1 depth starts at the normal close or fallback commit that made the tuple canonical, not at provisional landing.
P9	Equivocation in separate schedule windows slashes separate tranches; the global time watermark releases old liability without requiring proof progress.
P10	Maximum-prefix gas arithmetic has a legal block under admission caps; deterministic replay is identical; truncating a close block atomically removes its canonical effects.
L1-L6	The actual <code>lookahead()</code> path is deterministic, uses an L1-final snapshot, obeys <code>Q_MAX</code> , represents under-capacity as <code>VACANT</code> , and computes the exact six-address fill threshold.
L7-L10	Seeded placement preserves quotas over adversarial registries and seeds, gives no adjacent real builder inside a window, and pins the rejected 54.5% renormalized whale regression.

The settlement/fallback model currently reports 24 assertions; the lookahead model reports 187 assertions because it checks many registry/seed combinations. Neither models cryptography, EVM gas, proving latency or P2P availability; those remain measured gates in §11.